Setup All Project Dependencies

**This code may take some time to run, please be patient **

The following commands install the latest version of the packages.

In [1]:
```
# check python version and list the currently installed packages

!python --version
!pip list
```

```
Python 3.11.11
Package                                 Version
--------------------------------------- --------------
absl-py                                 2.1.0
accelerate                              0.34.2
adagio                                  0.2.6
aiobotocore                             2.20.0
aiohttp                                 3.9.5
aiohttp-cors                            0.7.0
aioitertools                            0.12.0
aiosignal                               1.3.2
aiosqlite                               0.19.0
alembic                                 1.15.1
altair                                  5.5.0
amazon-q-developer-jupyterlab-ext       3.4.7
amazon_sagemaker_jupyter_ai_q_developer 1.0.16
amazon_sagemaker_jupyter_scheduler      3.1.10
amazon-sagemaker-sql-editor             0.1.15
amazon-sagemaker-sql-execution          0.1.6
amazon-sagemaker-sql-magic              0.1.3
annotated-types                         0.7.0
ansi2html                               1.9.2
ansicolors                              1.1.8
antlr4-python3-runtime                  4.9.3
anyio                                   4.8.0
appdirs                                 1.4.4
archspec                                0.2.5
argon2-cffi                             23.1.0
argon2-cffi-bindings                    21.2.0
arrow                                   1.3.0
asn1crypto                              1.5.1
astroid                                 3.3.8
asttokens                               3.0.0
astunparse                              1.6.3
async-lru                               2.0.4
async-timeout                           4.0.3
attrs                                   23.2.0
autogluon                               1.2
autogluon.common                        1.2
autogluon.core                          1.2
autogluon.features                      1.2
autogluon.multimodal                    1.2
autogluon.tabular                       1.2
autogluon.timeseries                    1.2
autopep8                                2.0.4
autovizwidget                           0.22.0
```

aws-embedded-metrics	3.2.0
aws-glue-sessions	1.0.8
babel	2.17.0
bcrypt	4.3.0
beautifulsoup4	4.13.3
binaryornot	0.4.4
bleach	6.2.0
blinker	1.9.0
blis	1.0.1
boltons	24.0.0
boto3	1.36.23
botocore	1.36.23
Brotli	1.1.0
cached-property	1.5.2
cachetools	5.5.2
catalogue	2.0.10
catboost	1.2.7
certifi	2025.1.31
cffi	1.17.1
chardet	5.2.0
charset-normalizer	3.4.1
click	8.1.8
cloudpathlib	0.20.0
cloudpickle	3.1.1
colorama	0.4.6
colorful	0.5.6
colorlog	6.9.0
comm	0.2.2
conda	24.11.3
conda-libmamba-solver	24.9.0
conda-package-handling	2.4.0
conda_package_streaming	0.11.0
confection	0.1.5
contextlib2	21.6.0
contourpy	1.3.1
cookiecutter	2.6.0
coreforecast	0.0.12
croniter	1.4.1
cryptography	44.0.2
cycler	0.12.1
cymem	2.0.11
cytoolz	1.0.1
dash	2.18.1
dask	2025.2.0
databricks-sdk	0.44.1
dataclasses	0.8

dataclasses-json	0.6.7
datasets	2.2.1
debugpy	1.8.13
decorator	5.2.1
deepmerge	2.0
defusedxml	0.7.1
Deprecated	1.2.18
dill	0.3.9
diskcache	5.6.3
distlib	0.3.9
distributed	2025.2.0
distro	1.9.0
dnspython	2.7.0
docker	7.1.0
docstring-to-markdown	0.15
email_validator	2.2.0
entrypoints	0.4
evaluate	0.4.1
exceptiongroup	1.2.2
executing	2.1.0
faiss	1.9.0
fastai	2.7.18
fastapi	0.115.11
fastapi-cli	0.0.7
fastcore	1.7.20
fastdownload	0.0.7
fastjsonschema	2.21.1
fastprogress	1.0.3
filelock	3.17.0
flake8	7.1.2
Flask	3.1.0
flatbuffers	25.2.10
fonttools	4.56.0
fqdn	1.5.1
frozendict	2.4.6
frozenlist	1.5.0
fs	2.4.16
fsspec	2024.10.0
fugue	0.9.1
future	1.0.0
gast	0.6.0
gdown	5.2.0
git-remote-codecommit	1.16
gitdb	4.0.12
GitPython	3.1.44
gluonts	0.16.0

gmpy2	2.1.5
google-api-core	2.24.1
google-auth	2.38.0
google-pasta	0.2.0
googleapis-common-protos	1.69.0
graphene	3.4.3
graphql-core	3.2.6
graphql-relay	3.2.0
graphviz	0.20.3
greenlet	3.1.1
grpcio	1.62.2
gssapi	1.9.0
gunicorn	23.0.0
h11	0.14.0
h2	4.2.0
h5py	3.13.0
hdijupyterutils	0.22.0
hpack	4.1.0
httpcore	1.0.7
httptools	0.6.4
httpx	0.28.1
httpx-sse	0.4.0
huggingface_hub	0.29.1
hyperframe	6.1.0
hyperopt	0.2.7
idna	3.10
imagecodecs-lite	2019.12.3
imageio	2.37.0
importlib-metadata	6.10.0
importlib_resources	6.5.2
ipykernel	6.29.5
ipython	8.32.0
ipywidgets	8.1.5
isoduration	20.11.0
isort	6.0.1
itsdangerous	2.2.0
jedi	0.19.2
Jinja2	3.1.5
jmespath	1.0.1
joblib	1.4.2
json5	0.10.0
jsonpatch	1.33
jsonpath-ng	1.6.1
jsonpointer	3.0.0
jsonschema	4.23.0
jsonschema-specifications	2024.10.1

jupyter	1.1.1
jupyter-activity-monitor-extension	0.3.1
jupyter_ai	2.29.1
jupyter_ai_magics	2.29.1
jupyter_client	8.6.3
jupyter_collaboration	2.1.5
jupyter-console	6.6.3
jupyter_core	5.7.2
jupyter-dash	0.4.2
jupyter-events	0.12.0
jupyter-lsp	2.2.5
jupyter_scheduler	2.10.0
jupyter_server	2.15.0
jupyter_server_fileid	0.9.2
jupyter_server_mathjax	0.2.6
jupyter_server_proxy	4.4.0
jupyter_server_terminals	0.5.3
jupyter-ydoc	2.1.5
jupyterlab	4.3.5
jupyterlab_git	0.50.2
jupyterlab-lsp	5.0.3
jupyterlab_pygments	0.3.0
jupyterlab_server	2.27.3
jupyterlab_widgets	3.0.13
keras	3.8.0
kiwisolver	1.4.7
krb5	0.5.1
langchain	0.3.20
langchain-aws	0.2.10
langchain-community	0.3.19
langchain-core	0.3.41
langchain-text-splitters	0.3.6
langcodes	3.4.1
langsmith	0.2.11
language_data	1.3.0
lazy_loader	0.4
libmambapy	1.5.12
lightgbm	4.6.0
lightning	2.5.0.post0
lightning-utilities	0.13.1
linkify-it-py	2.0.3
llvmlite	0.44.0
locket	1.0.0
lxml	5.3.1
Mako	1.3.9
marisa-trie	1.2.1

Markdown	3.6
markdown-it-py	3.0.0
MarkupSafe	3.0.2
marshmallow	3.26.1
matplotlib	3.10.1
matplotlib-inline	0.1.7
mccabe	0.7.0
mdit-py-plugins	0.4.2
mdurl	0.1.2
memray	1.15.0
menuinst	2.2.0
mistune	3.1.2
ml-dtypes	0.4.0
mlflow	2.20.3
mlflow-skinny	2.20.3
mlforecast	0.13.4
mock	4.0.3
model-index	0.1.11
mpmath	1.3.0
msgpack	1.1.0
multidict	6.1.0
multiprocess	0.70.17
munkres	1.1.4
murmurhash	1.0.10
mypy_extensions	1.0.0
namex	0.0.8
narwhals	1.29.0
nbclient	0.10.2
nbconvert	7.16.6
nbdime	4.0.2
nbformat	5.10.4
nest_asyncio	1.6.0
networkx	3.4.2
nlpaug	1.1.11
nltk	3.9.1
nose	1.3.7
notebook	7.3.2
notebook_shim	0.2.4
numba	0.61.0
numpy	1.26.4
omegaconf	2.3.0
opencensus	0.11.3
opencensus-context	0.1.3
openmim	0.3.7
opentelemetry-api	1.30.0
opentelemetry-sdk	1.30.0

opentelemetry-semantic-conventions	0.51b0
opt_einsum	3.4.0
optree	0.14.1
optuna	4.2.1
ordered-set	4.1.0
orjson	3.10.15
overrides	7.7.0
packaging	24.2
pandas	2.2.3
pandocfilters	1.5.0
papermill	2.6.0
paramiko	3.5.1
parso	0.8.4
partd	1.4.2
pathos	0.3.3
patsy	1.0.1
pdf2image	1.17.0
pexpect	4.9.0
pickleshare	0.7.5
pillow	11.1.0
pip	24.3.1
pkgutil_resolve_name	1.3.10
platformdirs	4.3.6
plotly	6.0.0
pluggy	1.5.0
ply	3.11
pox	0.3.5
ppft	1.7.6.9
preshed	3.0.9
prometheus_client	0.21.1
prometheus_flask_exporter	0.23.1
prompt_toolkit	3.0.50
propcache	0.2.1
proto-plus	1.26.0
protobuf	4.25.3
psutil	5.9.8
ptyprocess	0.7.0
pure_eval	0.2.3
pure-sasl	0.6.2
py4j	0.10.9.9
pyarrow	17.0.0
pyasn1	0.6.1
pyasn1_modules	0.4.1
PyAthena	3.12.2
pycodestyle	2.12.1
pycosat	0.6.6

pycparser	2.22
pycrdt	0.12.8
pycrdt-websocket	0.15.4
pydantic	2.10.6
pydantic_core	2.27.2
pydantic-settings	2.8.1
pydocstyle	6.3.0
pyflakes	3.2.0
Pygments	2.19.1
PyHive	0.7.0
PyJWT	2.10.1
pylint	3.3.4
PyNaCl	1.5.0
pyOpenSSL	24.3.0
pyparsing	3.2.1
PySocks	1.7.1
pyspnego	0.11.2
pytesseract	0.3.10
python-dateutil	2.9.0.post0
python-dotenv	1.0.1
python-json-logger	2.0.7
python-lsp-jsonrpc	1.1.2
python-lsp-server	1.12.2
python-multipart	0.0.20
python-slugify	8.0.4
pytoolconfig	1.2.5
pytorch-lightning	2.5.0.post0
pytorch-metric-learning	2.3.0
pytz	2024.1
pyu2f	0.1.5
PyWavelets	1.8.0
PyYAML	6.0.2
pyzmq	26.2.1
querystring_parser	1.2.4
ray	2.37.0
redshift_connector	2.1.5
referencing	0.36.2
regex	2024.11.6
requests	2.32.3
requests-kerberos	0.15.0
requests-toolbelt	1.0.0
responses	0.18.0
retrying	1.3.4
rfc3339_validator	0.1.4
rfc3986-validator	0.1.1
rich	13.9.4

```
rich-toolkit                            0.11.3
rope                                    1.13.0
rpds-py                                 0.23.1
rsa                                     4.9
ruamel.yaml                             0.18.10
ruamel.yaml.clib                        0.2.8
s3fs                                    2024.10.0
s3transfer                              0.11.3
safetensors                             0.5.3
sagemaker                               2.240.0
sagemaker-core                          1.0.25
sagemaker-headless-execution-driver     0.0.13
sagemaker-jupyterlab-emr-extension      0.3.7
sagemaker-jupyterlab-extension          0.4.0
sagemaker-jupyterlab-extension-common   0.1.36
sagemaker-kernel-wrapper                0.0.5
sagemaker-mlflow                        0.1.0
sagemaker-studio-analytics-extension    0.1.5
sagemaker-studio-sparkmagic-lib         0.1.4
schema                                  0.7.7
scikit-image                            0.20.0
scikit-learn                            1.5.2
scipy                                   1.15.2
scramp                                  1.4.4
seaborn                                 0.13.2
Send2Trash                              1.8.3
sentencepiece                           0.1.99
seqeval                                 1.2.2
setproctitle                            1.3.5
setuptools                              75.8.2
shellingham                             1.5.4
simpervisor                             1.0.0
six                                     1.17.0
smart_open                              7.1.0
smdebug-rulesconfig                     1.0.1
smmap                                   5.0.2
sniffio                                 1.3.1
snowballstemmer                         2.2.0
snowflake-connector-python              3.13.2
sortedcontainers                        2.4.0
soupsieve                               2.5
spacy                                   3.8.2
spacy-legacy                            3.0.12
spacy-loggers                           1.0.5
sparkmagic                              0.21.0
SQLAlchemy                              2.0.38
```

sqlite-anyio	0.2.3
sqlparse	0.5.3
srsly	2.5.1
stack_data	0.6.3
starlette	0.46.0
statsforecast	1.7.8
statsmodels	0.14.4
supervisor	4.2.5
sympy	1.13.3
tabulate	0.9.0
tblib	3.0.0
tenacity	9.0.0
tensorboard	2.17.1
tensorboard_data_server	0.7.0
tensorboardX	2.6.2.2
tensorflow	2.17.0
tensorflow_estimator	2.15.0
termcolor	2.5.0
terminado	0.18.1
text-unidecode	1.3
textual	2.1.2
tf_keras	2.17.0
thinc	8.3.2
threadpoolctl	3.5.0
thrift	0.20.0
thrift_sasl	0.4.3
tifffile	2020.6.3
timm	1.0.3
tinycss2	1.4.0
tokenizers	0.21.0
tomli	2.2.1
tomlkit	0.13.2
toolz	0.12.1
torch	2.4.1.post100
torchmetrics	1.2.1
torchvision	0.19.1
tornado	6.4.2
tqdm	4.67.1
traitlets	5.14.3
transformers	4.49.0
triad	0.9.8
truststore	0.10.1
typer	0.15.2
typer-slim	0.15.2
types-python-dateutil	2.9.0.20241206
typing_extensions	4.12.2

```
typing_inspect                          0.9.0
typing_utils                            0.1.0
tzdata                                  2025.1
uc-micro-py                             1.0.3
ujson                                   5.10.0
unicodedata2                            16.0.0
uri-template                            1.3.0
urllib3                                 2.3.0
utilsforecast                           0.2.3
uvicorn                                 0.34.0
uvloop                                  0.21.0
virtualenv                              20.29.2
wasabi                                  1.1.3
watchfiles                              0.24.0
wcwidth                                 0.2.13
weasel                                  0.4.1
webcolors                               24.11.1
webencodings                            0.5.1
websocket-client                        1.8.0
websockets                              15.0
Werkzeug                                3.1.3
whatthepatch                            1.0.7
wheel                                   0.45.1
widgetsnbextension                      4.0.13
window_ops                              0.0.15
wrapt                                   1.17.2
xgboost                                 2.1.4
xxhash                                  3.5.0
y-py                                    0.6.2
yapf                                    0.43.0
yarl                                    1.18.3
ypy-websocket                           0.12.4
zict                                    3.0.0
zipp                                    3.21.0
zstandard                               0.19.0
```

In [2]:

```
# Upgrade pip and wrapt to the latest versions
!pip install --disable-pip-version-check -q pip --upgrade > /dev/null
!pip install --disable-pip-version-check -q wrapt --upgrade > /dev/null
```

AWS CLI and AWS Python SDK (boto3)

This installs the latest versions of AWS CLI, boto3 and botocore

In [3]:
```
!pip install --disable-pip-version-check -q awscli boto3 botocore
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
autogluon-multimodal 1.2 requires nvidia-ml-py3==7.352.0, which is not installed.
aiobotocore 2.20.0 requires botocore<1.36.24,>=1.36.20, but you have botocore 1.37.34 which is incompatible.
amazon-sagemaker-sql-magic 0.1.3 requires sqlparse==0.5.0, but you have sqlparse 0.5.3 which is incompatible.
autogluon-multimodal 1.2 requires jsonschema<4.22,>=4.18, but you have jsonschema 4.23.0 which is incompatible.
autogluon-multimodal 1.2 requires nltk<3.9,>=3.4.5, but you have nltk 3.9.1 which is incompatible.
autogluon-multimodal 1.2 requires omegaconf<2.3.0,>=2.1.1, but you have omegaconf 2.3.0 which is incompatible.
```

SageMaker Services

This section installs the latest version of sagemaker, smdebug, and sagemaker-experiments

In [15]:
```
!pip install --disable-pip-version-check -q sagemaker smdebug sagemaker-experiments
```

PyTorch

This section installs PyTorch using conda for compatibility

In [16]:
```
!conda install -y pytorch -c pytorch
```

```
Channels:
 - pytorch
 - conda-forge
Platform: linux-64
doneecting package metadata (repodata.json): -
^Clving environment: /
```

TensorFlow

Installs the latest version of TensorFlow

In [17]:
```
!pip install --disable-pip-version-check -q tensorflow
```

```
^C
ERROR: Operation cancelled by user
```

PyAthena

Installs the latest version of PyAthena

```
In [18]: !pip install --disable-pip-version-check -q PyAthena
```

AWS Data Wrangler

Installs the latest version of AWS Data Wrangler

```
In [19]: !pip install --disable-pip-version-check -q awswrangler
```

MatplotLib, Seaborn, Pandas

This installs the latest versions of MatPlotLib, Seaborn, Pandas

```
In [20]: !pip install --disable-pip-version-check -q matplotlib
         !pip install --disable-pip-version-check -q seaborn
         !pip install --disable-pip-version-check -q pandas
```

METAR Library

This will install the metar library which is used to read in raw formatted metar data in order to convert the values into columns

```
In [21]: !pip install --disable-pip-version-check -q metar
```

Windrose visualization library

This will install a library that aids in the development of wind rose visualizations showing where wind direction and speed are common.

```
In [1]: !pip install --disable-pip-version-check -q windrose
```

Checking Final versions of python and packages

```
In [23]: !python --version
         !pip list
```

```
Python 3.11.11
Package                                  Version
---------------------------------------- --------------
absl-py                                  2.1.0
accelerate                               0.34.2
adagio                                   0.2.6
aiobotocore                              2.20.0
aiohttp                                  3.9.5
aiohttp-cors                             0.7.0
aioitertools                             0.12.0
aiosignal                                1.3.2
aiosqlite                                0.19.0
alembic                                  1.15.1
altair                                   5.5.0
amazon-q-developer-jupyterlab-ext        3.4.7
amazon_sagemaker_jupyter_ai_q_developer 1.0.16
amazon_sagemaker_jupyter_scheduler       3.1.10
amazon-sagemaker-sql-editor              0.1.15
amazon-sagemaker-sql-execution           0.1.6
amazon-sagemaker-sql-magic               0.1.3
annotated-types                          0.7.0
ansi2html                                1.9.2
ansicolors                               1.1.8
antlr4-python3-runtime                   4.9.3
anyio                                    4.8.0
appdirs                                  1.4.4
archspec                                 0.2.5
argon2-cffi                              23.1.0
argon2-cffi-bindings                     21.2.0
arrow                                    1.3.0
asn1crypto                               1.5.1
astroid                                  3.3.8
asttokens                                3.0.0
astunparse                               1.6.3
async-lru                                2.0.4
async-timeout                            4.0.3
attrs                                    23.2.0
autogluon                                1.2
autogluon.common                         1.2
autogluon.core                           1.2
autogluon.features                       1.2
autogluon.multimodal                     1.2
autogluon.tabular                        1.2
autogluon.timeseries                     1.2
autopep8                                 2.0.4
autovizwidget                            0.22.0
```

aws-embedded-metrics	3.2.0
aws-glue-sessions	1.0.8
awscli	1.38.34
awswrangler	3.11.0
babel	2.17.0
bcrypt	4.3.0
beautifulsoup4	4.13.3
binaryornot	0.4.4
bleach	6.2.0
blinker	1.9.0
blis	1.0.1
boltons	24.0.0
boto3	1.37.34
botocore	1.37.34
Brotli	1.1.0
cached-property	1.5.2
cachetools	5.5.2
catalogue	2.0.10
catboost	1.2.7
certifi	2025.1.31
cffi	1.17.1
chardet	5.2.0
charset-normalizer	3.4.1
click	8.1.8
cloudpathlib	0.20.0
cloudpickle	3.1.1
colorama	0.4.6
colorful	0.5.6
colorlog	6.9.0
comm	0.2.2
conda	24.11.3
conda-libmamba-solver	24.9.0
conda-package-handling	2.4.0
conda_package_streaming	0.11.0
confection	0.1.5
contextlib2	21.6.0
contourpy	1.3.1
cookiecutter	2.6.0
coreforecast	0.0.12
croniter	1.4.1
cryptography	44.0.2
cycler	0.12.1
cymem	2.0.11
cytoolz	1.0.1
dash	2.18.1
dask	2025.2.0

databricks-sdk	0.44.1
dataclasses	0.8
dataclasses-json	0.6.7
datasets	2.2.1
debugpy	1.8.13
decorator	5.2.1
deepmerge	2.0
defusedxml	0.7.1
Deprecated	1.2.18
dill	0.3.9
diskcache	5.6.3
distlib	0.3.9
distributed	2025.2.0
distro	1.9.0
dnspython	2.7.0
docker	7.1.0
docstring-to-markdown	0.15
docutils	0.19
email_validator	2.2.0
entrypoints	0.4
evaluate	0.4.1
exceptiongroup	1.2.2
executing	2.1.0
faiss	1.9.0
fastai	2.7.18
fastapi	0.115.11
fastapi-cli	0.0.7
fastcore	1.7.20
fastdownload	0.0.7
fastjsonschema	2.21.1
fastprogress	1.0.3
filelock	3.17.0
flake8	7.1.2
Flask	3.1.0
flatbuffers	25.2.10
fonttools	4.56.0
fqdn	1.5.1
frozendict	2.4.6
frozenlist	1.5.0
fs	2.4.16
fsspec	2024.10.0
fugue	0.9.1
future	1.0.0
gast	0.6.0
gdown	5.2.0
git-remote-codecommit	1.16

gitdb	4.0.12
GitPython	3.1.44
gluonts	0.16.0
gmpy2	2.1.5
google-api-core	2.24.1
google-auth	2.38.0
google-pasta	0.2.0
googleapis-common-protos	1.69.0
graphene	3.4.3
graphql-core	3.2.6
graphql-relay	3.2.0
graphviz	0.20.3
greenlet	3.1.1
grpcio	1.62.2
gssapi	1.9.0
gunicorn	23.0.0
h11	0.14.0
h2	4.2.0
h5py	3.13.0
hdijupyterutils	0.22.0
hpack	4.1.0
httpcore	1.0.7
httptools	0.6.4
httpx	0.28.1
httpx-sse	0.4.0
huggingface_hub	0.29.1
hyperframe	6.1.0
hyperopt	0.2.7
idna	3.10
imagecodecs-lite	2019.12.3
imageio	2.37.0
importlib-metadata	6.10.0
importlib_resources	6.5.2
ipykernel	6.29.5
ipython	8.32.0
ipywidgets	8.1.5
isoduration	20.11.0
isort	6.0.1
itsdangerous	2.2.0
jedi	0.19.2
Jinja2	3.1.5
jmespath	1.0.1
joblib	1.4.2
json5	0.10.0
jsonpatch	1.33
jsonpath-ng	1.6.1

jsonpointer	3.0.0
jsonschema	4.23.0
jsonschema-specifications	2024.10.1
jupyter	1.1.1
jupyter-activity-monitor-extension	0.3.1
jupyter_ai	2.29.1
jupyter_ai_magics	2.29.1
jupyter_client	8.6.3
jupyter_collaboration	2.1.5
jupyter-console	6.6.3
jupyter_core	5.7.2
jupyter-dash	0.4.2
jupyter-events	0.12.0
jupyter-lsp	2.2.5
jupyter_scheduler	2.10.0
jupyter_server	2.15.0
jupyter_server_fileid	0.9.2
jupyter_server_mathjax	0.2.6
jupyter_server_proxy	4.4.0
jupyter_server_terminals	0.5.3
jupyter-ydoc	2.1.5
jupyterlab	4.3.5
jupyterlab_git	0.50.2
jupyterlab-lsp	5.0.3
jupyterlab_pygments	0.3.0
jupyterlab_server	2.27.3
jupyterlab_widgets	3.0.13
keras	3.8.0
kiwisolver	1.4.7
krb5	0.5.1
langchain	0.3.20
langchain-aws	0.2.10
langchain-community	0.3.19
langchain-core	0.3.41
langchain-text-splitters	0.3.6
langcodes	3.4.1
langsmith	0.2.11
language_data	1.3.0
lazy_loader	0.4
libmambapy	1.5.12
lightgbm	4.6.0
lightning	2.5.0.post0
lightning-utilities	0.13.1
linkify-it-py	2.0.3
llvmlite	0.44.0
locket	1.0.0

lxml	5.3.1
Mako	1.3.9
marisa-trie	1.2.1
Markdown	3.6
markdown-it-py	3.0.0
MarkupSafe	3.0.2
marshmallow	3.26.1
matplotlib	3.10.1
matplotlib-inline	0.1.7
mccabe	0.7.0
mdit-py-plugins	0.4.2
mdurl	0.1.2
memray	1.15.0
menuinst	2.2.0
metar	1.11.0
mistune	3.1.2
ml-dtypes	0.4.0
mlflow	2.20.3
mlflow-skinny	2.20.3
mlforecast	0.13.4
mock	4.0.3
model-index	0.1.11
mpmath	1.3.0
msgpack	1.1.0
multidict	6.1.0
multiprocess	0.70.17
munkres	1.1.4
murmurhash	1.0.10
mypy_extensions	1.0.0
namex	0.0.8
narwhals	1.29.0
nbclient	0.10.2
nbconvert	7.16.6
nbdime	4.0.2
nbformat	5.10.4
nest_asyncio	1.6.0
networkx	3.4.2
nlpaug	1.1.11
nltk	3.9.1
nose	1.3.7
notebook	7.3.2
notebook_shim	0.2.4
numba	0.61.0
numpy	1.26.4
omegaconf	2.3.0
opencensus	0.11.3

opencensus-context	0.1.3
openmim	0.3.7
opentelemetry-api	1.30.0
opentelemetry-sdk	1.30.0
opentelemetry-semantic-conventions	0.51b0
opt_einsum	3.4.0
optree	0.14.1
optuna	4.2.1
ordered-set	4.1.0
orjson	3.10.15
overrides	7.7.0
packaging	24.2
pandas	2.2.3
pandocfilters	1.5.0
papermill	2.6.0
paramiko	3.5.1
parso	0.8.4
partd	1.4.2
pathos	0.3.3
patsy	1.0.1
pdf2image	1.17.0
pexpect	4.9.0
pickleshare	0.7.5
pillow	11.1.0
pip	25.0.1
pkgutil_resolve_name	1.3.10
platformdirs	4.3.6
plotly	6.0.0
pluggy	1.5.0
ply	3.11
pox	0.3.5
ppft	1.7.6.9
preshed	3.0.9
prometheus_client	0.21.1
prometheus_flask_exporter	0.23.1
prompt_toolkit	3.0.50
propcache	0.2.1
proto-plus	1.26.0
protobuf	3.20.3
psutil	5.9.8
ptyprocess	0.7.0
pure_eval	0.2.3
pure-sasl	0.6.2
py4j	0.10.9.9
pyarrow	17.0.0
pyasn1	0.6.1

pyasn1_modules	0.4.1
PyAthena	3.12.2
pycodestyle	2.12.1
pycosat	0.6.6
pycparser	2.22
pycrdt	0.12.8
pycrdt-websocket	0.15.4
pydantic	2.10.6
pydantic_core	2.27.2
pydantic-settings	2.8.1
pydocstyle	6.3.0
pyflakes	3.2.0
Pygments	2.19.1
PyHive	0.7.0
pyinstrument	3.4.2
pyinstrument_cext	0.2.4
PyJWT	2.10.1
pylint	3.3.4
PyNaCl	1.5.0
pyOpenSSL	24.3.0
pyparsing	3.2.1
PySocks	1.7.1
pyspnego	0.11.2
pytesseract	0.3.10
python-dateutil	2.9.0.post0
python-dotenv	1.0.1
python-json-logger	2.0.7
python-lsp-jsonrpc	1.1.2
python-lsp-server	1.12.2
python-multipart	0.0.20
python-slugify	8.0.4
pytoolconfig	1.2.5
pytorch-lightning	2.5.0.post0
pytorch-metric-learning	2.3.0
pytz	2024.1
pyu2f	0.1.5
PyWavelets	1.8.0
PyYAML	6.0.2
pyzmq	26.2.1
querystring_parser	1.2.4
ray	2.37.0
redshift_connector	2.1.5
referencing	0.36.2
regex	2024.11.6
requests	2.32.3
requests-kerberos	0.15.0

requests-toolbelt	1.0.0
responses	0.18.0
retrying	1.3.4
rfc3339_validator	0.1.4
rfc3986-validator	0.1.1
rich	13.9.4
rich-toolkit	0.11.3
rope	1.13.0
rpds-py	0.23.1
rsa	4.7.2
ruamel.yaml	0.18.10
ruamel.yaml.clib	0.2.8
s3fs	2024.10.0
s3transfer	0.11.3
safetensors	0.5.3
sagemaker	2.240.0
sagemaker-core	1.0.25
sagemaker-experiments	0.1.45
sagemaker-headless-execution-driver	0.0.13
sagemaker-jupyterlab-emr-extension	0.3.7
sagemaker-jupyterlab-extension	0.4.0
sagemaker-jupyterlab-extension-common	0.1.36
sagemaker-kernel-wrapper	0.0.5
sagemaker-mlflow	0.1.0
sagemaker-studio-analytics-extension	0.1.5
sagemaker-studio-sparkmagic-lib	0.1.4
schema	0.7.7
scikit-image	0.20.0
scikit-learn	1.5.2
scipy	1.15.2
scramp	1.4.4
seaborn	0.13.2
Send2Trash	1.8.3
sentencepiece	0.1.99
seqeval	1.2.2
setproctitle	1.3.5
setuptools	75.8.2
shellingham	1.5.4
simpervisor	1.0.0
six	1.17.0
smart_open	7.1.0
smdebug	1.0.34
smdebug-rulesconfig	1.0.1
smmap	5.0.2
sniffio	1.3.1
snowballstemmer	2.2.0

snowflake-connector-python	3.13.2
sortedcontainers	2.4.0
soupsieve	2.5
spacy	3.8.2
spacy-legacy	3.0.12
spacy-loggers	1.0.5
sparkmagic	0.21.0
SQLAlchemy	2.0.38
sqlite-anyio	0.2.3
sqlparse	0.5.3
srsly	2.5.1
stack_data	0.6.3
starlette	0.46.0
statsforecast	1.7.8
statsmodels	0.14.4
supervisor	4.2.5
sympy	1.13.3
tabulate	0.9.0
tblib	3.0.0
tenacity	9.0.0
tensorboard	2.17.1
tensorboard_data_server	0.7.0
tensorboardX	2.6.2.2
tensorflow	2.17.0
tensorflow_estimator	2.15.0
termcolor	2.5.0
terminado	0.18.1
text-unidecode	1.3
textual	2.1.2
tf_keras	2.17.0
thinc	8.3.2
threadpoolctl	3.5.0
thrift	0.20.0
thrift_sasl	0.4.3
tifffile	2020.6.3
timm	1.0.3
tinycss2	1.4.0
tokenizers	0.21.0
tomli	2.2.1
tomlkit	0.13.2
toolz	0.12.1
torch	2.4.1.post100
torchmetrics	1.2.1
torchvision	0.19.1
tornado	6.4.2
tqdm	4.67.1

traitlets	5.14.3
transformers	4.49.0
triad	0.9.8
truststore	0.10.1
typer	0.15.2
typer-slim	0.15.2
types-python-dateutil	2.9.0.20241206
typing_extensions	4.12.2
typing_inspect	0.9.0
typing_utils	0.1.0
tzdata	2025.1
uc-micro-py	1.0.3
ujson	5.10.0
unicodedata2	16.0.0
uri-template	1.3.0
urllib3	2.3.0
utilsforecast	0.2.3
uvicorn	0.34.0
uvloop	0.21.0
virtualenv	20.29.2
wasabi	1.1.3
watchfiles	0.24.0
wcwidth	0.2.13
weasel	0.4.1
webcolors	24.11.1
webencodings	0.5.1
websocket-client	1.8.0
websockets	15.0
Werkzeug	3.1.3
whatthepatch	1.0.7
wheel	0.45.1
widgetsnbextension	4.0.13
window_ops	0.0.15
windrose	1.9.2
wrapt	1.17.2
xgboost	2.1.4
xxhash	3.5.0
y-py	0.6.2
yapf	0.43.0
yarl	1.18.3
ypy-websocket	0.12.4
zict	3.0.0
zipp	3.21.0
zstandard	0.19.0

In [24]:
```
# Mark the dependency setup as complete
setup_dependencies_passed = True
%store setup_dependencies_passed
%store
```

```
Stored 'setup_dependencies_passed' (bool)
Stored variables and their in-db values:
setup_dependencies_passed             -> True
setup_iam_roles_passed                -> True
setup_instance_check_passed           -> True
setup_s3_bucket_passed                -> True
```

Release the resources

Following cells will shut down your kernel in order to release resources

In [2]:
```
%%html

abs<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

abs

Shutting down your kernel for this notebook to release resources.

In [ ]:

Check Environment

In [1]:
```
import boto3

region = boto3.Session().region_name
session = boto3.session.Session()

ec2 = boto3.Session().client(service_name="ec2", region_name=region)
sm = boto3.Session().client(service_name="sagemaker", region_name=region)
```

In [2]:
```
import json

notebook_instance_name = None

try:
    with open("/opt/ml/metadata/resource-metadata.json") as notebook_info:
        data = json.load(notebook_info)
        domain_id = data["DomainId"]
        resource_arn = data["ResourceArn"]
        region = resource_arn.split(":")[3]
        name = data["ResourceName"]
    print("DomainId: {}".format(domain_id))
    print("Name: {}".format(name))
except:
    print("+++++++++++++++++++++++++++++++++++++++++")
    print("[ERROR]: COULD NOT RETRIEVE THE METADATA.")
    print("+++++++++++++++++++++++++++++++++++++++++")
```

```
DomainId: d-dlu97fb10zsw
Name: default
```

In [3]:
```
describe_domain_response = sm.describe_domain(DomainId=domain_id)
print(describe_domain_response["Status"])
```

```
InService
```

In [4]:
```
try:
    get_status_response = sm.get_sagemaker_servicecatalog_portfolio_status()
    print(get_status_response["Status"])
except:
    pass
```

```
Enabled
```

In [5]:
```python
if (
    describe_domain_response["Status"] == "InService"
    and get_status_response["Status"] == "Enabled"
    and "default" in name
):
    setup_instance_check_passed = True
    print("[OK] Checks passed!  Great Job!!  Please Continue.")
else:
    setup_instance_check_passed = False
    print("+++++++++++++++++++++++++++++++++++++++++++++++++")
    print("[ERROR]: WE HAVE IDENTIFIED A MISCONFIGURATION.")
    print(describe_domain_response["Status"])
    print(get_status_response["Status"])
    print(name)
    print("+++++++++++++++++++++++++++++++++++++++++++++++++")
```

```
[OK] Checks passed!  Great Job!!  Please Continue.
```

In [6]:
```python
print(setup_instance_check_passed)
```

```
True
```

In [7]:
```python
%store setup_instance_check_passed
```

```
Stored 'setup_instance_check_passed' (bool)
```

In [8]:
```python
%store
```

```
Stored variables and their in-db values:
setup_dependencies_passed              -> True
setup_iam_roles_passed                 -> True
setup_instance_check_passed            -> True
setup_s3_bucket_passed                 -> True
```

In [1]:
```
%%html

<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
```

```
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:

```

```

Create S3 buckets for stages of the project

In order to establish these buckets in your own sagemaker environment, simply change the intials at the end of each bucket to your unique initials.

In [1]:
```
# import libraries
import boto3
import sagemaker

from botocore.client import ClientError
```

```
/opt/conda/lib/python3.11/site-packages/pydantic/_internal/_fields.py:192: UserWarning: Field name "json" in "MonitoringDatasetFo
rmat" shadows an attribute in parent "Base"
  warnings.warn(
sagemaker.config INFO - Not applying SDK defaults from location: /etc/xdg/sagemaker/config.yaml
sagemaker.config INFO - Not applying SDK defaults from location: /home/sagemaker-user/.config/sagemaker/config.yaml
```

Establish session and client information

In [2]:
```
session = boto3.session.Session()
region = session.region_name
s3 = boto3.Session().client(service_name="s3", region_name=region)
```

Create Raw data S3 bucket

In [3]:
```
#set env variable
%env RAW_BUCKET=group9-ml-proj-raw-data-bucket-to

raw_bucket = "group9-ml-proj-raw-data-bucket-to"

try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=raw_bucket)
    else:
        s3.create_bucket(Bucket=raw_bucket, CreateBucketConfiguration={"LocationConstraint": region})
    print("Raw Data Bucket created:", raw_bucket)
except Exception as e:
    print("Raw Data Bucket may already exist or an error occurred:", e)
```

```
env: RAW_BUCKET=group9-ml-proj-raw-data-bucket-grw
Raw Data Bucket created: group9-ml-proj-raw-data-bucket-grw
```

In [4]:
```
%%bash
aws s3 ls s3://$RAW_BUCKET/
```

Create Processed data bucket

In [5]:
```
#set env variable
%env PROCESSED_BUCKET=group9-ml-proj-processed-data-bucket-

processed_bucket = "group9-ml-proj-processed-data-bucket-grw"

try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=processed_bucket)
    else:
        s3.create_bucket(Bucket=processed_bucket, CreateBucketConfiguration={"LocationConstraint": region})
    print("Processed Data Bucket created:", processed_bucket)
except Exception as e:
    print("Processed Data Bucket may already exist or an error occurred:", e)
```

```
env: PROCESSED_BUCKET=group9-ml-proj-processed-data-bucket-grw
Processed Data Bucket created: group9-ml-proj-processed-data-bucket-grw
```

In [6]:
```
%%bash
aws s3 ls s3://$PROCESSED_BUCKET/
```

Athena Query results bucket

In [ ]:
```
#set env variable
%env QUERY_BUCKET=group9-ml-proj-athena-query-bucket-grw

query_bucket = "group9-ml-proj-athena-query-bucket-grw"

try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=query_bucket)
    else:
        s3.create_bucket(Bucket=query_bucket, CreateBucketConfiguration={"LocationConstraint": region})
    print("Athena Query Bucket created:", query_bucket)
```

```
except Exception as e:
    print("Athena Query Bucket may already exist or an error occurred:", e)
```

In [ ]:
```
%%bash
aws s3 ls s3://$QUERY_BUCKET/
```

Create Training Artifacts bucket

In [7]:
```
#set env variable
%env TNG_ART_BUCKET=group9-ml-proj-training-artifacts-bucket-grw

tng_art_bucket = "group9-ml-proj-training-artifacts-bucket-grw"

try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=tng_art_bucket)
    else:
        s3.create_bucket(Bucket=tng_art_bucket, CreateBucketConfiguration={"LocationConstraint": region})
    print("Training Artifacts Bucket created:", tng_art_bucket)
except Exception as e:
    print("Training Artifacts Bucket may already exist or an error occurred:", e)
```

```
env: TNG_ART_BUCKET=group9-ml-proj-training-artifacts-bucket-grw
Training Artifacts Bucket created: group9-ml-proj-training-artifacts-bucket-grw
```

In [8]:
```
%%bash
aws s3 ls s3://$TNG_ART_BUCKET/
```

Create Model Artifacts Bucket

In [9]:
```
#set env variable
%env MONITORING_BUCKET=group9-ml-proj-model-artifacts-bucket-grw

model_art_bucket = "group9-ml-proj-model-artifacts-bucket-grw"

try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=model_art_bucket)
    else:
        s3.create_bucket(Bucket=model_art_bucket, CreateBucketConfiguration={"LocationConstraint": region})
    print("Model Artifacts Bucket created:", model_art_bucket)
```

```
except Exception as e:
    print("Model Artifacts Bucket may already exist or an error occurred:", e)
```

```
env: MONITORING_BUCKET=group9-ml-proj-model-artifacts-bucket-grw
Model Artifacts Bucket created: group9-ml-proj-model-artifacts-bucket-grw
```

In [10]:

```
%%bash
aws s3 ls s3://$MODEL_ART_BUCKET/
```

```
2025-03-24 17:48:48 group9-ml-proj-model-artifacts-bucket-grw
2025-03-24 17:48:46 group9-ml-proj-processed-data-bucket-grw
data-bucket-grw8:45 group9-ml-proj-raw-
2025-03-24 17:48:47 group9-ml-proj-training-artifacts-bucket-grw
 sagemaker-studio-7sb8gxpljq9
2025-02-28 21:29:38 sagemaker-studio-chxawrkbs1q
agemaker-studio-o4i4ukj5y3b
2025-02-28 21:29:40 sagemaker-us-east-1-804823187130
```

Create Monitoring Bucket

In [11]:

```
#set env variable
%env MONITORING_BUCKET=group9-ml-proj-monitoring-bucket-grw

monitoring_bucket = "group9-ml-proj-monitoring-bucket-grw"

try:
    if region == "us-east-1":
        s3.create_bucket(Bucket=monitoring_bucket)
    else:
        s3.create_bucket(Bucket=monitoring_bucket, CreateBucketConfiguration={"LocationConstraint": region})
    print("Inference/Monitoring Bucket created:", monitoring_bucket)
except Exception as e:
    print("Inference/Monitoring Bucket may already exist or an error occurred:", e)
```

```
env: MONITORING_BUCKET=group9-ml-proj-monitoring-bucket-grw
Inference/Monitoring Bucket created: group9-ml-proj-monitoring-bucket-grw
```

In [12]:

```
%%bash
aws s3 ls s3://$MONITORING_BUCKET/
```

In [1]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button
```

```
<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:

```
```

Athena Database and Table Creation

This notebook will establish the AWS Athena Database that will be responsible for developing the raw_data and processed_data tables which will store the data.

In [1]:
```
# import libraries
import boto3
import time
```

In [2]:
```
# run athena query function
def run_athena_query(query, database, output_location):
    athena_client = boto3.client('athena')
    # query execution.
    response = athena_client.start_query_execution(
        QueryString=query,
        QueryExecutionContext={'Database': database},
        ResultConfiguration={'OutputLocation': output_location}
    )
    query_execution_id = response['QueryExecutionId']
    print(f"Query submitted. Execution ID: {query_execution_id}")
    # poll query until it completes.
    state = 'RUNNING'
    while state in ['RUNNING', 'QUEUED']:
        response = athena_client.get_query_execution(QueryExecutionId=query_execution_id)
        state = response['QueryExecution']['Status']['State']
        print(f"Current query state: {state}")
        if state in ['RUNNING', 'QUEUED']:
            time.sleep(2)

    if state == 'FAILED':
        reason = response['QueryExecution']['Status'].get('StateChangeReason', 'Unknown error')
        raise Exception(f"Query {query_execution_id} failed: {reason}")

    print(f"Query {query_execution_id} completed successfully.")
    return query_execution_id
```

In [3]:
```
# set athena database name
athena_database = 'processed_data'
# set bucket used by Athena to store query results.
output_location = 's3://group9-ml-proj-athena-query-bucket-grw/'  # change your initials
```

```python
# create db query
create_db_query = f"CREATE DATABASE IF NOT EXISTS {athena_database};"
print("Creating database...")
run_athena_query(create_db_query, database='default', output_location=output_location)
```

```
Creating database...
Query submitted. Execution ID: 1f46bdd2-bf96-4e2d-a0b7-8802aa7ee779
Current query state: QUEUED
Current query state: SUCCEEDED
Query 1f46bdd2-bf96-4e2d-a0b7-8802aa7ee779 completed successfully.
```

Out[3]: '1f46bdd2-bf96-4e2d-a0b7-8802aa7ee779'

In [4]:

```python
# create external table
create_table_query = f"""
CREATE EXTERNAL TABLE IF NOT EXISTS {athena_database}.processed_data (
    FL_DATE STRING,
    CRS_DEP_TIME STRING,
    OP_UNIQUE_CARRIER STRING,
    DEST STRING,
    wind_dir_degrees DOUBLE,
    wind_speed_kt DOUBLE,
    wind_gust_kt DOUBLE,
    visibility_statute_mi DOUBLE,
    temperature_c DOUBLE,
    dewpoint_c DOUBLE,
    altimeter_hpa DOUBLE,
    Flight_Status STRING,
    Carrier_AA BOOLEAN,
    Carrier_AS BOOLEAN,
    Carrier_B6 BOOLEAN,
    Carrier_DL BOOLEAN,
    Carrier_F9 BOOLEAN,
    Carrier_G4 BOOLEAN,
    Carrier_HA BOOLEAN,
    Carrier_NK BOOLEAN,
    Carrier_OO BOOLEAN,
    Carrier_UA BOOLEAN,
    Carrier_WN BOOLEAN,
    DEST_REGION STRING,
    Dest_Region_Midwest BOOLEAN,
    Dest_Region_Mountain BOOLEAN,
    Dest_Region_Northeast BOOLEAN,
    Dest_Region_OCONUS BOOLEAN,
    Dest_Region_South BOOLEAN,
    Dest_Region_Southwest BOOLEAN,
```

```
    Dest_Region_West BOOLEAN,
    Flight_Status_Binary BOOLEAN
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
  'separatorChar' = ',',
  'quoteChar' = '\"'
)
LOCATION 's3://group9-ml-proj-processed-data-bucket-grw/all_data/'
TBLPROPERTIES ('skip.header.line.count'='1');
"""
print("Creating external table...")
run_athena_query(create_table_query, database=athena_database, output_location=output_location)

print("Athena database and table setup completed successfully.")
```

```
Creating external table...
Query submitted. Execution ID: e4362e9e-e2bb-4220-a981-aa1d309964aa
Current query state: QUEUED
Current query state: SUCCEEDED
Query e4362e9e-e2bb-4220-a981-aa1d309964aa completed successfully.
Athena database and table setup completed successfully.
```

In [1]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:

Weather Cleaning and Organization Notebook

This notebook will focus on converting the raw METAR and SPECI weather data into one usable data frame and exporting it to a CSV file that will be used to perform predictive analysis for on time departures out of San Diego International Airport. This finalized notebook will be used to perform a merge with the on time flight dataset resulting in weather data points for each of the flight information that was obtained.

In [1]:
```
# Import Libraries
import pandas as pd
import re

from datetime import datetime
from metar import Metar
```

In [2]:
```
# Import the data files
df1 = pd.read_csv('raw_weather/KSAN_2023_wx.csv')
df2 = pd.read_csv('raw_weather/KSAN_2024_wx.csv')
df3 = pd.read_csv('raw_weather/KSAN_2025_wx.csv')

# create list of dfs
dfs = [df1, df2, df3]
```

```
/tmp/ipykernel_5660/2189777474.py:2: DtypeWarning: Columns (34,35,36,42,46,50,79) have mixed types. Specify dtype option on impor
t or set low_memory=False.
  df1 = pd.read_csv('raw_weather/KSAN_2023_wx.csv')
/tmp/ipykernel_5660/2189777474.py:3: DtypeWarning: Columns (2,36,41) have mixed types. Specify dtype option on import or set low_
memory=False.
  df2 = pd.read_csv('raw_weather/KSAN_2024_wx.csv')
```

In [3]:
```
# Removing unnecessary rows besides REM which houses the raw METAR data and concatenating the dataframes
metar_dfs = []

for i, df in enumerate(dfs, start=1):
    if 'REM' in df.columns:
        metar_dfs.append(df[['REM']])
    else:
        print('REM column not found in df{i}')

# concatenating the dfs
comb_metar_df = pd.concat(metar_dfs, ignore_index=True)
```

In [4]:
```
comb_metar_df.head()
```

Out[4]:

	REM
0	SYN08672290 12366 8//06 10161 20128 30148 4016...
1	MET10512/31/22 16:10:03 SPECI KSAN 010010Z 180...
2	MET12712/31/22 16:51:03 METAR KSAN 010051Z 160...
3	MET12612/31/22 17:51:03 METAR KSAN 010151Z 150...
4	MET10512/31/22 18:16:03 SPECI KSAN 010216Z 140...

In [5]:

```
comb_metar_df.tail()
```

Out[5]:

	REM
30326	MET14103/06/25 21:01:01 SPECI KSAN 070501Z 330...
30327	MET13803/06/25 21:13:01 SPECI KSAN 070513Z 350...
30328	MET16903/06/25 21:51:01 METAR KSAN 070551Z 350...
30329	MET11903/06/25 22:51:01 METAR KSAN 070651Z 330...
30330	SOD77324 HR PRECIPITATION (IN): 0.59 WEATHER(C...

In [6]:

```
# Filter so that the data only contains METAR data and not extra synthetic information that does not conform to the standard META
comb_metar_df = comb_metar_df[comb_metar_df['REM'].str.startswith('MET', na=False)]
```

In [7]:

```
# reset the index of the dataframe
comb_metar_df.reset_index(drop=True, inplace=True)
```

In [8]:

```
print(comb_metar_df.head())
print(comb_metar_df.tail())
```

```
                                                   REM
0  MET10512/31/22 16:10:03 SPECI KSAN 010010Z 180...
1  MET12712/31/22 16:51:03 METAR KSAN 010051Z 160...
2  MET12612/31/22 17:51:03 METAR KSAN 010151Z 150...
3  MET10512/31/22 18:16:03 SPECI KSAN 010216Z 140...
4  MET10812/31/22 18:38:03 SPECI KSAN 010238Z 140...
                                                       REM
24583  MET19203/06/25 20:51:01 METAR KSAN 070451Z 350...
24584  MET14103/06/25 21:01:01 SPECI KSAN 070501Z 330...
24585  MET13803/06/25 21:13:01 SPECI KSAN 070513Z 350...
24586  MET16903/06/25 21:51:01 METAR KSAN 070551Z 350...
24587  MET11903/06/25 22:51:01 METAR KSAN 070651Z 330...
```

In [9]:

```
# establish a prefix pattern to extract the information before the actual beginning of the METAR data

prefix_pattern = re.compile(
    # the MET pattern and 3 digit code that we do not need
    r'^MET\d{3}'
    # local month
    r'(?P<month>\d{2})/'
    # local day
    r'(?P<day>\d{2})/'
    #local year
    r'(?P<year>\d{2})\s+'
    # local hour
    r'(?P<hour>\d{2}):'
    # local minute
    r'(?P<minute>\d{2}):'
    # local second
    r'(?P<second>\d{2})\s+'
    # rest of the actual metar
    r'(?P<metar>.*)$'
)
```

In [10]:

```
# create function that will parse the prefix data, dropping the MET and the three digit code but making a date and time column
def parse_prefix(line):
    match = prefix_pattern.match(line)
    if not match:
        return None

    local_date = f"{match.group('month')}/{match.group('day')}/{match.group('year')}"
    local_time = f"{match.group('hour')}:{match.group('minute')}:{match.group('second')}"
    metar = match.group('metar')

    return {
```

```
        'local_date': local_date,
        'local_time': local_time,
        'metar': metar
    }
```

In [11]:

```
# create function that will parse the raw METAR data.
def parse_metar(metar):
    try:
        parsed = Metar.Metar(metar)

        return {
            'station_id': parsed.station_id,
            'wind_dir_degrees': parsed.wind_dir.value() if parsed.wind_dir else None,
            'wind_speed_kt': parsed.wind_speed.value() if parsed.wind_speed else None,
            'wind_gust_kt': parsed.wind_gust.value() if parsed.wind_gust else None,
            'visibility_statute_mi': parsed.vis.value() if parsed.vis else None,
            'temperature_c': parsed.temp.value() if parsed.temp else None,
            'dewpoint_c': parsed.dewpt.value() if parsed.dewpt else None,
            'altimeter_hpa': parsed.press.value() if parsed.press else None,
        }
    except Exception as e:
        return None
```

In [12]:

```
# tie all the functions together in order to break up each field of every METAR observation into their own columns

parsed_metar_data = []

# process each line of the comb_metar_df
for line in comb_metar_df['REM']:
    prefix_data = parse_prefix(line)
    if prefix_data is None:
        continue

    metar_data = parse_metar(prefix_data['metar'])
    if metar_data is None:
        continue

    combined_record = {**prefix_data, **metar_data}
    parsed_metar_data.append(combined_record)

# create the new df from parsed records
organized_metar_df = pd.DataFrame(parsed_metar_data)
organized_metar_df.reset_index(drop=True, inplace=True)
```

In [13]:
```
print(organized_metar_df.head())
```

```
  local_date local_time                                              metar  \
0   12/31/22   16:10:03  SPECI KSAN 010010Z 18004KT 10SM SCT009 BKN026 ...
1   12/31/22   16:51:03  METAR KSAN 010051Z 16006KT 8SM -RA SCT008 BKN0...
2   12/31/22   17:51:03  METAR KSAN 010151Z 15006KT 10SM FEW008 BKN021 ...
3   12/31/22   18:16:03  SPECI KSAN 010216Z 14010KT 10SM FEW009 BKN031 ...
4   12/31/22   18:38:03  SPECI KSAN 010238Z 14010G21KT 10SM FEW009 BKN0...

  station_id  wind_dir_degrees  wind_speed_kt  wind_gust_kt  \
0       KSAN             180.0            4.0           NaN
1       KSAN             160.0            6.0           NaN
2       KSAN             150.0            6.0           NaN
3       KSAN             140.0           10.0           NaN
4       KSAN             140.0           10.0          21.0

   visibility_statute_mi  temperature_c  dewpoint_c  altimeter_hpa
0                   10.0           15.6        12.8          30.01
1                    8.0           15.6        13.3          30.00
2                   10.0           15.6        12.8          29.98
3                   10.0           15.6        12.8          29.97
4                   10.0           15.6        12.8          29.96
```

In [14]:
```
# drop the metar column
organized_metar_df.drop(columns=['metar'], inplace=True)
print(organized_metar_df.head())
```

```
  local_date local_time station_id  wind_dir_degrees  wind_speed_kt  \
0   12/31/22   16:10:03       KSAN             180.0            4.0
1   12/31/22   16:51:03       KSAN             160.0            6.0
2   12/31/22   17:51:03       KSAN             150.0            6.0
3   12/31/22   18:16:03       KSAN             140.0           10.0
4   12/31/22   18:38:03       KSAN             140.0           10.0

   wind_gust_kt  visibility_statute_mi  temperature_c  dewpoint_c  \
0           NaN                   10.0           15.6        12.8
1           NaN                    8.0           15.6        13.3
2           NaN                   10.0           15.6        12.8
3           NaN                   10.0           15.6        12.8
4          21.0                   10.0           15.6        12.8

   altimeter_hpa
0          30.01
1          30.00
2          29.98
3          29.97
4          29.96
```

In [15]:

```
# replace all NaN values in the wind_gust_kt column with 0
organized_metar_df.fillna({'wind_gust_kt':0}, inplace=True)
print(organized_metar_df.head())
```

```
  local_date local_time station_id  wind_dir_degrees  wind_speed_kt  \
0   12/31/22   16:10:03       KSAN             180.0            4.0
1   12/31/22   16:51:03       KSAN             160.0            6.0
2   12/31/22   17:51:03       KSAN             150.0            6.0
3   12/31/22   18:16:03       KSAN             140.0           10.0
4   12/31/22   18:38:03       KSAN             140.0           10.0

   wind_gust_kt  visibility_statute_mi  temperature_c  dewpoint_c  \
0           0.0                   10.0           15.6        12.8
1           0.0                    8.0           15.6        13.3
2           0.0                   10.0           15.6        12.8
3           0.0                   10.0           15.6        12.8
4          21.0                   10.0           15.6        12.8

   altimeter_hpa
0          30.01
1          30.00
2          29.98
3          29.97
4          29.96
```

In [16]:

```
# output the data frame to a csv
organized_metar_df.to_csv('raw_weather/organized_metar_data.csv', index=False)
```

In [17]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:

```
```

Raw Weather S3 Bucket Upload

The purpose of this notebook is to upload the rawest organized metar data into the raw_data S3 bucket. We will map this to an AWS Glue database in a later notebook.

In [1]:
```
# import libraries
import boto3
import os
```

In [2]:
```
# examine list of buckets available
s3 = boto3.client('s3')
response = s3.list_buckets()
print("Existing buckets:")
for bucket in response['Buckets']:
    print(bucket['Name'])
```

```
Existing buckets:
group9-ml-proj-model-artifacts-bucket-grw
group9-ml-proj-monitoring-bucket-grw
group9-ml-proj-processed-data-bucket-grw
group9-ml-proj-raw-data-bucket-grw
group9-ml-proj-training-artifacts-bucket-grw
sagemaker-studio-7sb8gxpljq9
sagemaker-studio-chxawrkbs1q
sagemaker-studio-o4i4ukj5y3b
sagemaker-us-east-1-804823187130
```

In [3]:
```
# set local path of organized weather CSV file
local_file_path = 'raw_weather/organized_metar_data.csv'
```

In [4]:
```
# set the raw data bucket variable
raw_bucket = "group9-ml-proj-raw-data-bucket-grw"
```

In [5]:
```
# set the S3 key path, we are using flat structure because this will only hold three files
s3_key = "data/organized_metar_data.csv"
```

In [6]:
```
# upload file to the S3 bucket
try:
    s3.upload_file(local_file_path, raw_bucket, s3_key)
    print(f"Uploaded {local_file_path} to s3://{raw_bucket}/{s3_key}")
```

```
except Exception as e:
    print("Error uploading file to S3:", e)
```

```
Uploaded raw_weather/organized_metar_data.csv to s3://group9-ml-proj-raw-data-bucket-grw/data/organized_metar_data.csv
```

Shutdown the kernel

In [1]:
```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:
```
```

Flight Cleaning Notebook

This notebook will be responsible for merging all the flight information for the San Diego International Airport's On Time performance covering two years.

In [1]:
```python
# import libraries
import pandas as pd
```

In [2]:
```python
# import the csv files
Jan23 = pd.read_csv('raw_on-time/Jan23_OT_report.csv')
Feb23 = pd.read_csv('raw_on-time/Feb23_OT_report.csv')
Mar23 = pd.read_csv('raw_on-time/Mar23_OT_report.csv')
Apr23 = pd.read_csv('raw_on-time/Apr23_OT_report.csv')
May23 = pd.read_csv('raw_on-time/May23_OT_report.csv')
Jun23 = pd.read_csv('raw_on-time/Jun23_OT_report.csv')
Jul23 = pd.read_csv('raw_on-time/Jul23_OT_report.csv')
Aug23 = pd.read_csv('raw_on-time/Aug23_OT_report.csv')
Sep23 = pd.read_csv('raw_on-time/Sep23_OT_report.csv')
Oct23 = pd.read_csv('raw_on-time/Oct23_OT_report.csv')
Nov23 = pd.read_csv('raw_on-time/Nov23_OT_report.csv')
Dec23 = pd.read_csv('raw_on-time/Dec23_OT_report.csv')
Jan24 = pd.read_csv('raw_on-time/Jan24_OT_report.csv')
Feb24 = pd.read_csv('raw_on-time/Feb24_OT_report.csv')
Mar24 = pd.read_csv('raw_on-time/Mar24_OT_report.csv')
Apr24 = pd.read_csv('raw_on-time/Apr24_OT_report.csv')
May24 = pd.read_csv('raw_on-time/May24_OT_report.csv')
Jun24 = pd.read_csv('raw_on-time/Jun24_OT_report.csv')
Jul24 = pd.read_csv('raw_on-time/Jul24_OT_report.csv')
Aug24 = pd.read_csv('raw_on-time/Aug24_OT_report.csv')
Sep24 = pd.read_csv('raw_on-time/Sep24_OT_report.csv')
Oct24 = pd.read_csv('raw_on-time/Oct24_OT_report.csv')
Nov24 = pd.read_csv('raw_on-time/Nov24_OT_report.csv')
Dec24 = pd.read_csv('raw_on-time/Dec24_OT_report.csv')
```

In [3]:
```python
# list the df's in chronological order
dfs = [Jan23, Feb23, Mar23, Apr23, May23, Jun23, Jul23, Aug23, Sep23, Oct23, Nov23, Dec23,
       Jan24, Feb24, Mar24, Apr24, May24, Jun24, Jul24, Aug24, Sep24, Oct24, Nov24, Dec24]
```

In [4]:
```python
# merge df's
merged_df = pd.concat(dfs, ignore_index=True)
```

In [5]:
```
# Filter all df so that only flights leaving San Diego are represented
SAN_df = merged_df.loc[merged_df['ORIGIN'] == 'SAN']
# Reset the index
SAN_df = SAN_df.reset_index(drop=True)
```

In [6]:
```
# Get shape of the df
SAN_df.shape
```

Out[6]:
```
(185600, 18)
```

In [7]:
```
# get the data types of the df
SAN_df.dtypes
```

Out[7]:
```
FL_DATE                 object
OP_UNIQUE_CARRIER       object
OP_CARRIER_FL_NUM        int64
ORIGIN                  object
DEST                    object
CRS_DEP_TIME             int64
DEP_TIME               float64
DEP_DEL15              float64
DEP_DELAY_GROUP        float64
DEP_TIME_BLK            object
CANCELLED              float64
CANCELLATION_CODE       object
CARRIER_DELAY          float64
WEATHER_DELAY          float64
NAS_DELAY              float64
SECURITY_DELAY         float64
LATE_AIRCRAFT_DELAY    float64
DEP_DELAY              float64
dtype: object
```

In [8]:
```
print(SAN_df.head())
```

```
               FL_DATE OP_UNIQUE_CARRIER  OP_CARRIER_FL_NUM ORIGIN DEST  \
0  1/1/2023 12:00:00 AM                AA               1055    SAN  DFW
1  1/1/2023 12:00:00 AM                AA               1651    SAN  CLT
2  1/1/2023 12:00:00 AM                AA               1663    SAN  ORD
3  1/1/2023 12:00:00 AM                AA               1765    SAN  DFW
4  1/1/2023 12:00:00 AM                AA               1939    SAN  DFW

   CRS_DEP_TIME  DEP_TIME  DEP_DEL15  DEP_DELAY_GROUP DEP_TIME_BLK  CANCELLED  \
0           715     713.0        0.0             -1.0    0700-0759        0.0
1           615     644.0        1.0              1.0    0600-0659        0.0
2          2259    2250.0        0.0             -1.0    2200-2259        0.0
3           815     810.0        0.0             -1.0    0800-0859        0.0
4          1450    1447.0        0.0             -1.0    1400-1459        0.0

  CANCELLATION_CODE  CARRIER_DELAY  WEATHER_DELAY  NAS_DELAY  SECURITY_DELAY  \
0               NaN            NaN            NaN        NaN             NaN
1               NaN            NaN            NaN        NaN             NaN
2               NaN            NaN            NaN        NaN             NaN
3               NaN            NaN            NaN        NaN             NaN
4               NaN            NaN            NaN        NaN             NaN

   LATE_AIRCRAFT_DELAY  DEP_DELAY
0                  NaN        NaN
1                  NaN        NaN
2                  NaN        NaN
3                  NaN        NaN
4                  NaN        NaN
```

In [9]:

```
# adjust the FL_DATE column to datetime
SAN_df['FL_DATE'] = pd.to_datetime(SAN_df['FL_DATE']).dt.date
```

```
/tmp/ipykernel_5818/1251424886.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling ba
ck to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
  SAN_df['FL_DATE'] = pd.to_datetime(SAN_df['FL_DATE']).dt.date
```

In [10]:

```
# convert CRS_DEP_TIME to a string
SAN_df['CRS_DEP_TIME'] = SAN_df['CRS_DEP_TIME'].astype(str).str.zfill(4)
# convert CRS_DEP_TIME to time format
SAN_df['CRS_DEP_TIME'] = pd.to_datetime(SAN_df['CRS_DEP_TIME'], format='%H%M').dt.time
```

In [11]:

```
print(SAN_df.head())
```

```
     FL_DATE OP_UNIQUE_CARRIER  OP_CARRIER_FL_NUM ORIGIN DEST CRS_DEP_TIME  \
0  2023-01-01                AA               1055    SAN  DFW     07:15:00
1  2023-01-01                AA               1651    SAN  CLT     06:15:00
2  2023-01-01                AA               1663    SAN  ORD     22:59:00
3  2023-01-01                AA               1765    SAN  DFW     08:15:00
4  2023-01-01                AA               1939    SAN  DFW     14:50:00

  DEP_TIME  DEP_DEL15  DEP_DELAY_GROUP DEP_TIME_BLK  CANCELLED  \
0     713.0        0.0             -1.0    0700-0759        0.0
1     644.0        1.0              1.0    0600-0659        0.0
2    2250.0        0.0             -1.0    2200-2259        0.0
3     810.0        0.0             -1.0    0800-0859        0.0
4    1447.0        0.0             -1.0    1400-1459        0.0

  CANCELLATION_CODE  CARRIER_DELAY  WEATHER_DELAY  NAS_DELAY  SECURITY_DELAY  \
0               NaN            NaN            NaN        NaN             NaN
1               NaN            NaN            NaN        NaN             NaN
2               NaN            NaN            NaN        NaN             NaN
3               NaN            NaN            NaN        NaN             NaN
4               NaN            NaN            NaN        NaN             NaN

  LATE_AIRCRAFT_DELAY  DEP_DELAY
0                 NaN        NaN
1                 NaN        NaN
2                 NaN        NaN
3                 NaN        NaN
4                 NaN        NaN
```

In [12]:
```python
# create function to handle missing DEP_TIME for cancelled flights
def convert_dep_time(val):
    if pd.isnull(val):
        return None
    try:
        val_float = float(val)
        val_int = int(val_float)
        time_str = str(val_int).zfill(4)
        return pd.to_datetime(time_str, format='%H%M').time()
    except Exception:
        return None
```

In [13]:
```python
# apply the function to the DEP_TIME column
SAN_df['DEP_TIME'] = SAN_df['DEP_TIME'].apply(convert_dep_time)
```

In [14]:
```python
print(SAN_df.head())
```

```
     FL_DATE OP_UNIQUE_CARRIER  OP_CARRIER_FL_NUM ORIGIN DEST CRS_DEP_TIME  \
0  2023-01-01                AA               1055    SAN  DFW     07:15:00
1  2023-01-01                AA               1651    SAN  CLT     06:15:00
2  2023-01-01                AA               1663    SAN  ORD     22:59:00
3  2023-01-01                AA               1765    SAN  DFW     08:15:00
4  2023-01-01                AA               1939    SAN  DFW     14:50:00

   DEP_TIME  DEP_DEL15  DEP_DELAY_GROUP DEP_TIME_BLK  CANCELLED  \
0  07:13:00        0.0             -1.0    0700-0759        0.0
1  06:44:00        1.0              1.0    0600-0659        0.0
2  22:50:00        0.0             -1.0    2200-2259        0.0
3  08:10:00        0.0             -1.0    0800-0859        0.0
4  14:47:00        0.0             -1.0    1400-1459        0.0

  CANCELLATION_CODE  CARRIER_DELAY  WEATHER_DELAY  NAS_DELAY  SECURITY_DELAY  \
0               NaN            NaN            NaN        NaN             NaN
1               NaN            NaN            NaN        NaN             NaN
2               NaN            NaN            NaN        NaN             NaN
3               NaN            NaN            NaN        NaN             NaN
4               NaN            NaN            NaN        NaN             NaN

   LATE_AIRCRAFT_DELAY  DEP_DELAY
0                  NaN        NaN
1                  NaN        NaN
2                  NaN        NaN
3                  NaN        NaN
4                  NaN        NaN
```

In [15]:
```
# convert the DEP_DEL15 column to int
SAN_df['DEP_DEL15'] = SAN_df['DEP_DEL15'].fillna(0).astype(int)
```

In [16]:
```
# convert CANCELLED column to int
SAN_df['CANCELLED'] = SAN_df['CANCELLED'].fillna(0).astype(int)
```

In [17]:
```
# check what values are present in cancellation_code
SAN_df['CANCELLATION_CODE'].value_counts()
```

Out[17]:
```
CANCELLATION_CODE
A    992
B    902
C    222
D      2
Name: count, dtype: int64
```

In [18]:
```
# fill missing values with None in CANCELLATION_CODE
SAN_df['CANCELLATION_CODE'] = SAN_df['CANCELLATION_CODE'].fillna('None')
```

In [19]:
```
# check what values are present in CARRIER_DELAY
SAN_df['CARRIER_DELAY'].value_counts()
```

Out[19]:
```
CARRIER_DELAY
0.0       23180
1.0         887
2.0         871
3.0         776
4.0         722
          ...
263.0         1
1055.0        1
436.0         1
863.0         1
284.0         1
Name: count, Length: 546, dtype: int64
```

In [20]:
```
# convert the CARRIER_DELAY column to int
SAN_df['CARRIER_DELAY'] = SAN_df['CARRIER_DELAY'].fillna(0).astype(int)
```

In [21]:
```
# convert the WEATHER_DELAY column to int
SAN_df['WEATHER_DELAY'] = SAN_df['WEATHER_DELAY'].fillna(0).astype(int)
```

In [22]:
```
# convert the NAS_DELAY column to int
SAN_df['NAS_DELAY'] = SAN_df['NAS_DELAY'].fillna(0).astype(int)
```

In [23]:
```
# convert the SECURITY_DELAY column to int
SAN_df['SECURITY_DELAY'] = SAN_df['SECURITY_DELAY'].fillna(0).astype(int)
```

In [24]:
```
# convert the LATE_AIRCRAFT_DELAY column to int
SAN_df['LATE_AIRCRAFT_DELAY'] = SAN_df['LATE_AIRCRAFT_DELAY'].fillna(0).astype(int)
```

In [25]:
```
# reorder the columns so times are next to the date
SAN_df = SAN_df[['FL_DATE', 'CRS_DEP_TIME', 'DEP_TIME', 'OP_UNIQUE_CARRIER', 'OP_CARRIER_FL_NUM', 'ORIGIN', 'DEST', 'DEP_DEL15',
print(SAN_df.head())
```

```
     FL_DATE CRS_DEP_TIME  DEP_TIME OP_UNIQUE_CARRIER  OP_CARRIER_FL_NUM  \
0  2023-01-01     07:15:00  07:13:00                AA               1055
1  2023-01-01     06:15:00  06:44:00                AA               1651
2  2023-01-01     22:59:00  22:50:00                AA               1663
3  2023-01-01     08:15:00  08:10:00                AA               1765
4  2023-01-01     14:50:00  14:47:00                AA               1939

  ORIGIN DEST  DEP_DEL15  CANCELLED CANCELLATION_CODE  CARRIER_DELAY  \
0    SAN  DFW          0          0              None              0
1    SAN  CLT          1          0              None              0
2    SAN  ORD          0          0              None              0
3    SAN  DFW          0          0              None              0
4    SAN  DFW          0          0              None              0

   WEATHER_DELAY  NAS_DELAY  SECURITY_DELAY  LATE_AIRCRAFT_DELAY
0              0          0               0                    0
1              0          0               0                    0
2              0          0               0                    0
3              0          0               0                    0
4              0          0               0                    0
```

In [26]:

```python
# Save the df to a csv file
SAN_df.to_csv('raw_on-time/SAN_OT_report.csv', index=False)
```

In [27]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

Raw Flight Data Merging and Uploading to S3

In [1]:
```
# import libraries
import boto3
import os
```

In [2]:
```
# examine list of buckets available
s3 = boto3.client('s3')
response = s3.list_buckets()
print("Existing buckets:")
for bucket in response['Buckets']:
    print(bucket['Name'])
```

```
Existing buckets:
group9-ml-proj-model-artifacts-bucket-grw
group9-ml-proj-monitoring-bucket-grw
group9-ml-proj-processed-data-bucket-grw
group9-ml-proj-raw-data-bucket-grw
group9-ml-proj-training-artifacts-bucket-grw
sagemaker-studio-7sb8gxpljq9
sagemaker-studio-chxawrkbs1q
sagemaker-studio-o4i4ukj5y3b
sagemaker-us-east-1-804823187130
```

In [3]:
```
# set local path of merged CSV file
local_file_path = 'raw_on-time/SAN_OT_report.csv'
```

In [4]:
```
# set the raw data bucket variable
raw_bucket = "group9-ml-proj-raw-data-bucket-grw"
```

In [5]:
```
# set the S3 key path, we are using flat structure because this will only hold two files
s3_key = "data/SAN_OT_report.csv"
```

In [6]:
```
# upload file to the S3 bucket
try:
    s3.upload_file(local_file_path, raw_bucket, s3_key)
    print(f"Uploaded {local_file_path} to s3://{raw_bucket}/{s3_key}")
except Exception as e:
    print("Error uploading file to S3:", e)
```

```
Uploaded raw_on-time/SAN_OT_report.csv to s3://group9-ml-proj-raw-data-bucket-grw/data/SAN_OT_report.csv
```

Shutdown the kernel

In [1]:
```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:
```
```

Merged Notebook

This notebook will serve the purpose of merging the organized METAR and San Diego On Time report into one usable dataset.

The merge will focus on merging on the variables local_date and local_time from the organized_metar_data.csv file and flight_date and CRS_DEP_TIME from the SAN_OT_report.csv file. In the event that the local_time and CRS_DEP_TIME do not exactly match the closest weather observation prior to CRS_DEP_TIME will be used.

In [2]:
```
# import libraries
import pandas as pd
```

In [3]:
```
# read in the data
wx_data = pd.read_csv("/home/sagemaker-user/ADS-508-Project/2_raw_data/raw_weather/organized_metar_data.csv")
flt_data = pd.read_csv("/home/sagemaker-user/ADS-508-Project/2_raw_data/raw_on-time/SAN_OT_report.csv")
```

In [4]:
```
# create a datetime column for the weather data
wx_data['wx_datetime'] = pd.to_datetime(wx_data['local_date'] + ' ' + wx_data['local_time'],
                                        format="%m/%d/%y %H:%M:%S",
                                        errors='coerce')

wx_data[['local_date', 'local_time', 'wx_datetime']].head()
```

Out[4]:

	local_date	local_time	wx_datetime
0	12/31/22	16:10:03	2022-12-31 16:10:03
1	12/31/22	16:51:03	2022-12-31 16:51:03
2	12/31/22	17:51:03	2022-12-31 17:51:03
3	12/31/22	18:16:03	2022-12-31 18:16:03
4	12/31/22	18:38:03	2022-12-31 18:38:03

In [5]:
```
# create a datetime column for the flight data
flt_data['flt_datetime'] = pd.to_datetime(flt_data['FL_DATE'] + ' ' + flt_data['CRS_DEP_TIME'],
                                          format="%Y-%m-%d %H:%M:%S",
                                          errors='coerce')

flt_data[['FL_DATE', 'CRS_DEP_TIME', 'flt_datetime']].head()
```

Out[5]:

	FL_DATE	CRS_DEP_TIME	flt_datetime
0	2023-01-01	07:15:00	2023-01-01 07:15:00
1	2023-01-01	06:15:00	2023-01-01 06:15:00
2	2023-01-01	22:59:00	2023-01-01 22:59:00
3	2023-01-01	08:15:00	2023-01-01 08:15:00
4	2023-01-01	14:50:00	2023-01-01 14:50:00

In [6]:

```
# sort the flight data by flt_datetime
flt_data = flt_data.sort_values(by='flt_datetime')
```

In [7]:

```
# check both sorted by created datetime fields
print("METAR is monotonic:", wx_data["wx_datetime"].is_monotonic_increasing)
print("Flights are monotonic:", flt_data["flt_datetime"].is_monotonic_increasing)
```

```
METAR is monotonic: True
Flights are monotonic: True
```

In [8]:

```
# merge the dataframes on the datetime fields created prior
merged_data = pd.merge_asof(flt_data,
                           wx_data,left_on='flt_datetime',
                           right_on='wx_datetime',
                           direction='backward')

merged_data.head()
```

Out[8]:

	FL_DATE	CRS_DEP_TIME	DEP_TIME	OP_UNIQUE_CARRIER	OP_CARRIER_FL_NUM	ORIGIN	DEST	DEP_DEL15	CANCELLED	CANCELLATION
0	2023-01-01	06:10:00	06:05:00	DL	977	SAN	DTW	0	0	
1	2023-01-01	06:15:00	06:17:00	DL	2423	SAN	SLC	0	0	
2	2023-01-01	06:15:00	06:44:00	AA	1651	SAN	CLT	1	0	
3	2023-01-01	06:15:00	06:15:00	DL	820	SAN	ATL	0	0	
4	2023-01-01	06:15:00	06:13:00	DL	914	SAN	MSP	0	0	

5 rows × 27 columns

In [9]:

```
# check the column names
merged_data.columns
```

Out[9]:

```
Index(['FL_DATE', 'CRS_DEP_TIME', 'DEP_TIME', 'OP_UNIQUE_CARRIER',
       'OP_CARRIER_FL_NUM', 'ORIGIN', 'DEST', 'DEP_DEL15', 'CANCELLED',
       'CANCELLATION_CODE', 'CARRIER_DELAY', 'WEATHER_DELAY', 'NAS_DELAY',
       'SECURITY_DELAY', 'LATE_AIRCRAFT_DELAY', 'flt_datetime', 'local_date',
       'local_time', 'station_id', 'wind_dir_degrees', 'wind_speed_kt',
       'wind_gust_kt', 'visibility_statute_mi', 'temperature_c', 'dewpoint_c',
       'altimeter_hpa', 'wx_datetime'],
      dtype='object')
```

In [10]:

```
# drop unnecessary columns
merged_data = merged_data.drop(columns=['local_date', 'local_time', 'wx_datetime', 'station_id', 'flt_datetime', 'ORIGIN'])
```

In [11]:

```
# save the merged data
merged_data.to_csv("/home/sagemaker-user/ADS-508-Project/2_raw_data/raw_merged/raw_merged_data.csv", index=False)
```

In [12]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
```

```
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:

```

```

Upload Raw Merged Dataset to S3

In [1]:
```
# import libraries
import boto3
import os
```

In [2]:
```
# examine list of buckets available
s3 = boto3.client('s3')
response = s3.list_buckets()
print("Existing buckets:")
for bucket in response['Buckets']:
    print(bucket['Name'])
```

```
Existing buckets:
group9-ml-proj-model-artifacts-bucket-grw
group9-ml-proj-monitoring-bucket-grw
group9-ml-proj-processed-data-bucket-grw
group9-ml-proj-raw-data-bucket-grw
group9-ml-proj-training-artifacts-bucket-grw
sagemaker-studio-7sb8gxpljq9
sagemaker-studio-chxawrkbs1q
sagemaker-studio-o4i4ukj5y3b
sagemaker-us-east-1-804823187130
```

In [3]:
```
# set local path of merged CSV file
local_file_path = 'raw_merged/raw_merged_data.csv'
```

In [4]:
```
# set the raw data bucket variable
raw_bucket = "group9-ml-proj-raw-data-bucket-grw"
```

In [5]:
```
# set the S3 key path, we are using flat structure because this will only hold two files
s3_key = "data/raw_merged_data.csv"
```

In [6]:
```
# upload file to the S3 bucket
try:
    s3.upload_file(local_file_path, raw_bucket, s3_key)
    print(f"Uploaded {local_file_path} to s3://{raw_bucket}/{s3_key}")
except Exception as e:
    print("Error uploading file to S3:", e)
```

```
Uploaded raw_merged/raw_merged_data.csv to s3://group9-ml-proj-raw-data-bucket-grw/data/raw_merged_data.csv
```

In [1]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

In [ ]:

```
```

Exploratory Data Analysis Section

This notebook will use PyAthena to query a AWS Glue database table that contains the raw information then perform Exploratory Data Analysis.

In [2]:
```
# import libraries
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import os
import boto3

from windrose import WindroseAxes
```

In [3]:
```
# read in the csv data file into a pandas dataframe
raw_df = pd.read_csv("../2_raw_data/raw_merged/raw_merged_data.csv")
```

In [4]:
```
# Get shape of the data
raw_df.shape
```

Out[4]:
```
(185600, 21)
```

In [5]:
```
# Get a list of the column names
raw_df.columns
```

Out[5]:
```
Index(['FL_DATE', 'CRS_DEP_TIME', 'DEP_TIME', 'OP_UNIQUE_CARRIER',
       'OP_CARRIER_FL_NUM', 'DEST', 'DEP_DEL15', 'CANCELLED',
       'CANCELLATION_CODE', 'CARRIER_DELAY', 'WEATHER_DELAY', 'NAS_DELAY',
       'SECURITY_DELAY', 'LATE_AIRCRAFT_DELAY', 'wind_dir_degrees',
       'wind_speed_kt', 'wind_gust_kt', 'visibility_statute_mi',
       'temperature_c', 'dewpoint_c', 'altimeter_hpa'],
      dtype='object')
```

In [6]:
```
# get information on the df
raw_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 185600 entries, 0 to 185599
Data columns (total 21 columns):
 #   Column                Non-Null Count   Dtype  
---  ------                --------------   -----  
 0   FL_DATE               185600 non-null  object 
 1   CRS_DEP_TIME          185600 non-null  object 
 2   DEP_TIME              183548 non-null  object 
 3   OP_UNIQUE_CARRIER     185600 non-null  object 
 4   OP_CARRIER_FL_NUM     185600 non-null  int64  
 5   DEST                  185600 non-null  object 
 6   DEP_DEL15             185600 non-null  int64  
 7   CANCELLED             185600 non-null  int64  
 8   CANCELLATION_CODE     2118 non-null    object 
 9   CARRIER_DELAY         185600 non-null  int64  
 10  WEATHER_DELAY         185600 non-null  int64  
 11  NAS_DELAY             185600 non-null  int64  
 12  SECURITY_DELAY        185600 non-null  int64  
 13  LATE_AIRCRAFT_DELAY   185600 non-null  int64  
 14  wind_dir_degrees      170031 non-null  float64
 15  wind_speed_kt         185600 non-null  float64
 16  wind_gust_kt          185600 non-null  float64
 17  visibility_statute_mi  185600 non-null  float64
 18  temperature_c         185600 non-null  float64
 19  dewpoint_c            185600 non-null  float64
 20  altimeter_hpa         185600 non-null  float64
dtypes: float64(7), int64(8), object(6)
memory usage: 29.7+ MB
```

In [7]:
```
# get number of nulls in each column
raw_df.isnull().sum()
```

Out[7]:
```
FL_DATE                    0
CRS_DEP_TIME               0
DEP_TIME                2052
OP_UNIQUE_CARRIER          0
OP_CARRIER_FL_NUM          0
DEST                       0
DEP_DEL15                  0
CANCELLED                  0
CANCELLATION_CODE     183482
CARRIER_DELAY              0
WEATHER_DELAY              0
NAS_DELAY                  0
SECURITY_DELAY             0
LATE_AIRCRAFT_DELAY        0
wind_dir_degrees       15569
wind_speed_kt              0
wind_gust_kt               0
visibility_statute_mi      0
temperature_c              0
dewpoint_c                 0
altimeter_hpa              0
dtype: int64
```

In [8]:
```
# get summary statistics of the dataframe
raw_df.describe()
```

Out[8]:

	OP_CARRIER_FL_NUM	DEP_DEL15	CANCELLED	CARRIER_DELAY	WEATHER_DELAY	NAS_DELAY	SECURITY_DELAY	LATE_AIRCR
count	185600.000000	185600.000000	185600.000000	185600.000000	185600.000000	185600.000000	185600.000000	185
mean	1992.292236	0.095264	0.011412	3.595609	0.565388	3.030501	0.022328	
std	1281.470818	0.293580	0.106214	33.489037	12.527305	15.422100	1.125169	
min	6.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	894.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
50%	1939.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
75%	2924.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
max	8784.000000	1.000000	1.000000	3300.000000	1171.000000	1431.000000	276.000000	2

Target Variable Exploration of Delayed and Cancelled flights

In [9]:
```
# count the number of DEP_DEL15 values
raw_df['DEP_DEL15'].value_counts()
```

Out[9]:
```
DEP_DEL15
0    167919
1     17681
Name: count, dtype: int64
```

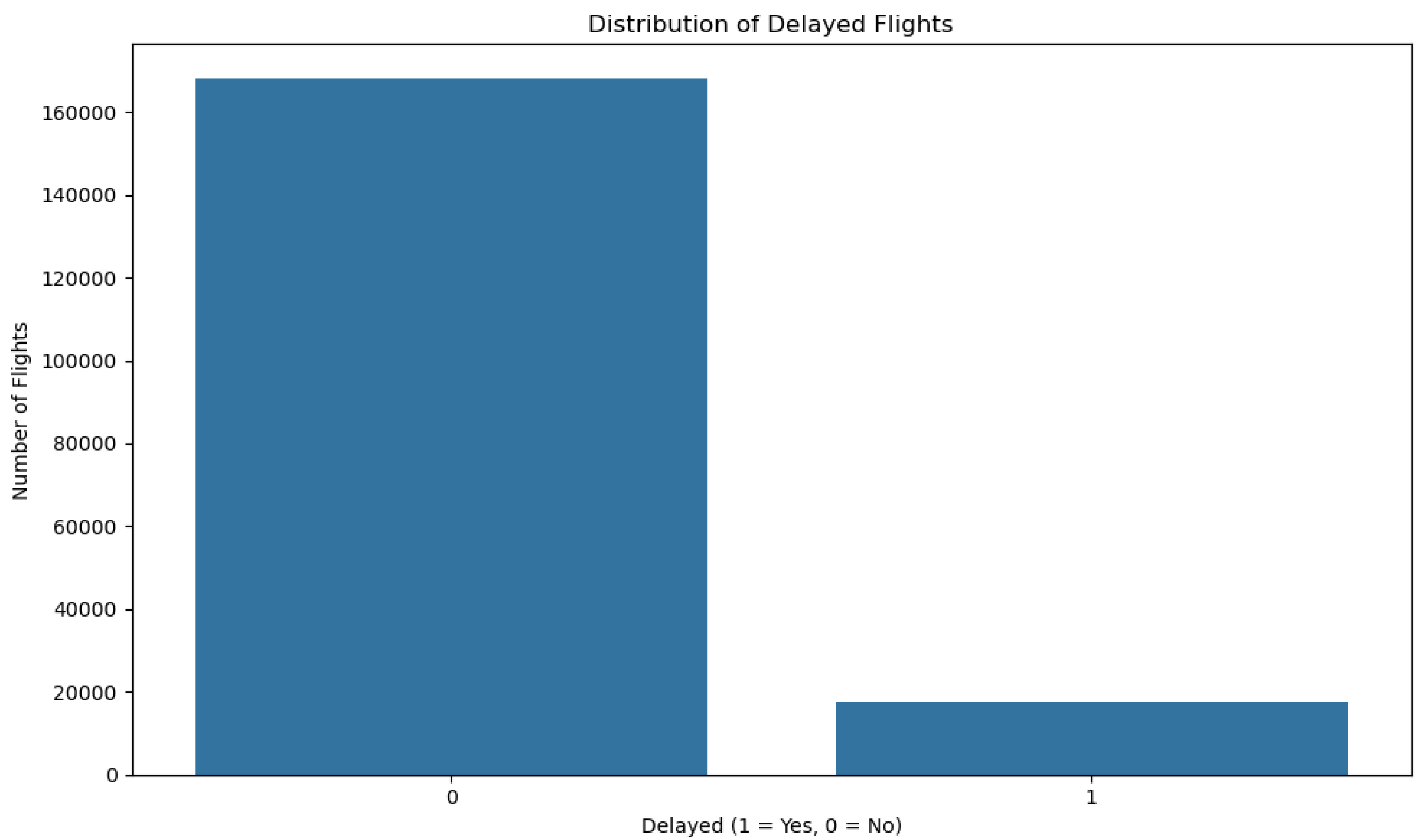
In [10]:
```
# get unique values of DEP_DEL15
raw_df['DEP_DEL15'].unique()
```

Out[10]:
```
array([0, 1])
```

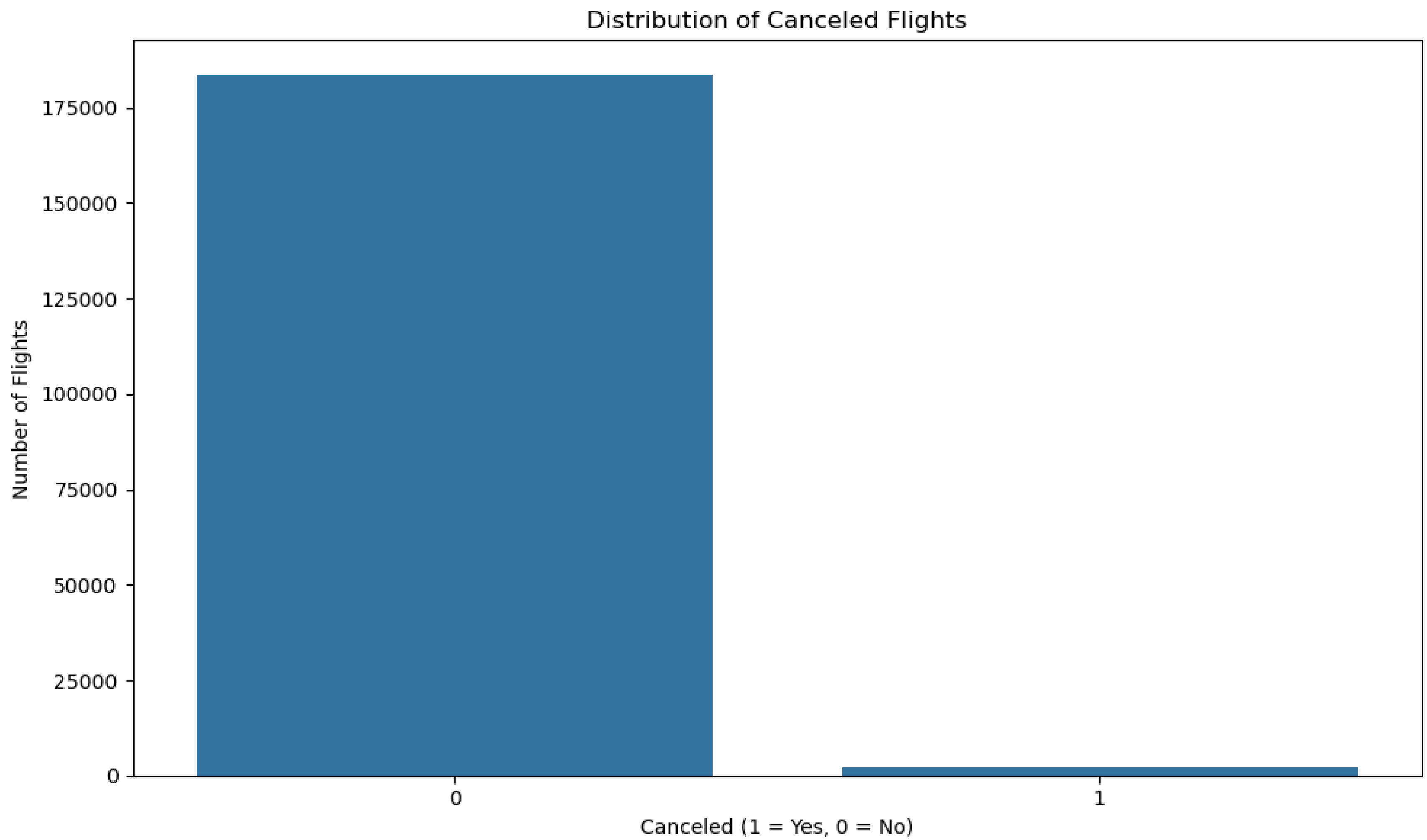
In [11]:
```
# count the number of CANCELLED values
raw_df['CANCELLED'].value_counts()
```

Out[11]:
```
CANCELLED
0    183482
1      2118
Name: count, dtype: int64
```

In [12]:
```
# visualize the distribution of delayed flights
plt.figure(figsize=(10,6))
sns.countplot(data=raw_df, x='DEP_DEL15')
plt.title('Distribution of Delayed Flights')
plt.xlabel('Delayed (1 = Yes, 0 = No)')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()
```



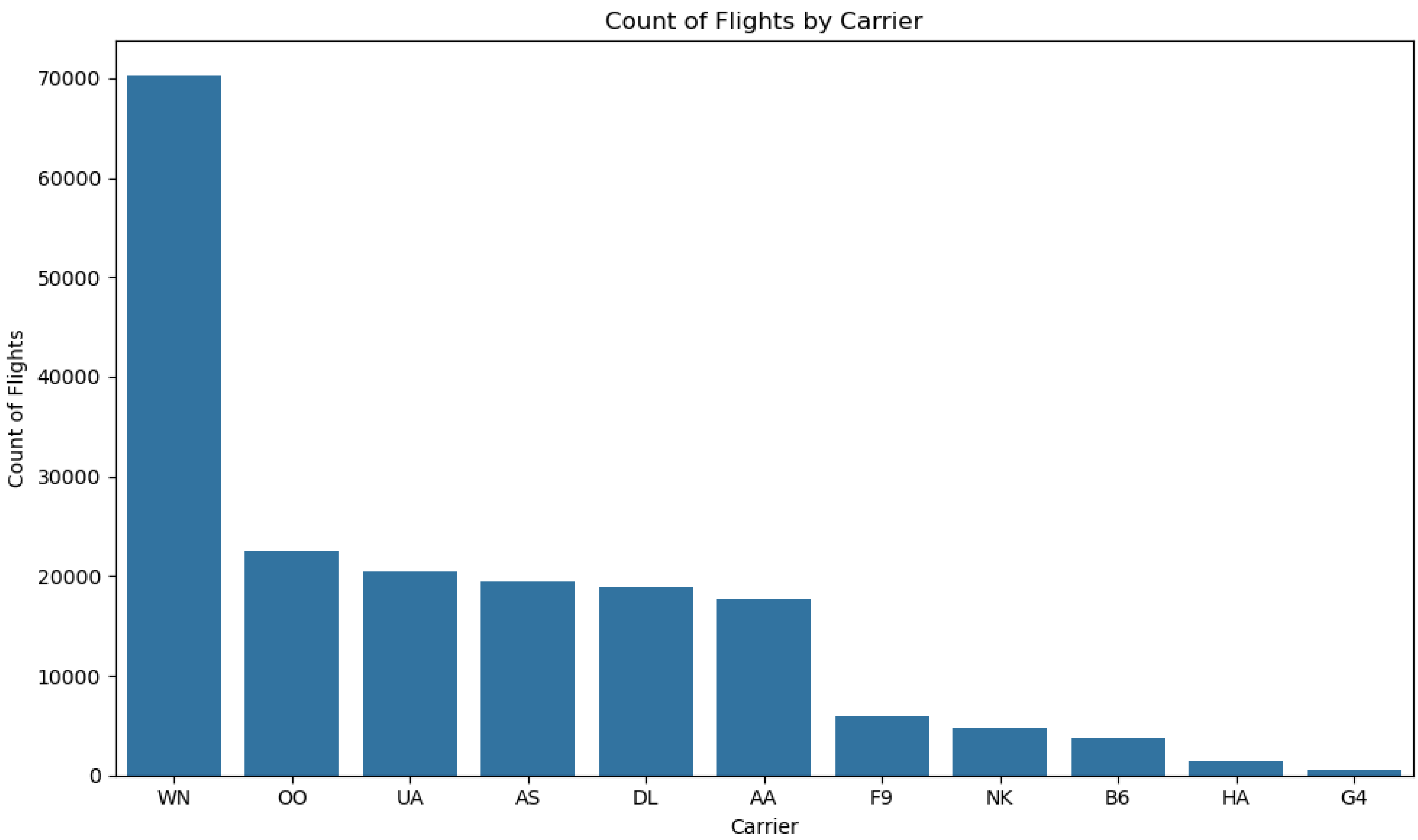
In [13]:
```
# visualize distribution of canceled flights
plt.figure(figsize=(10,6))
sns.countplot(data=raw_df, x='CANCELLED')
plt.title('Distribution of Canceled Flights')
plt.xlabel('Canceled (1 = Yes, 0 = No)')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()
```



In [14]:

```
# explore the distribution of carriers and flights
# get count of flights by airline
carrier_counts = raw_df['OP_UNIQUE_CARRIER'].value_counts().reset_index()
carrier_counts.columns = ['Carrier', 'Count of Flights']

# plot the count of flights by airline
plt.figure(figsize=(10,6))
sns.barplot(data=carrier_counts, x='Carrier', y='Count of Flights')
plt.title('Count of Flights by Carrier')
plt.xlabel('Carrier')
plt.ylabel('Count of Flights')
```

```
plt.tight_layout()
plt.show()
```



In [15]:
```
# create new category in dataframe for eda
raw_eda_df = raw_df.copy()
```

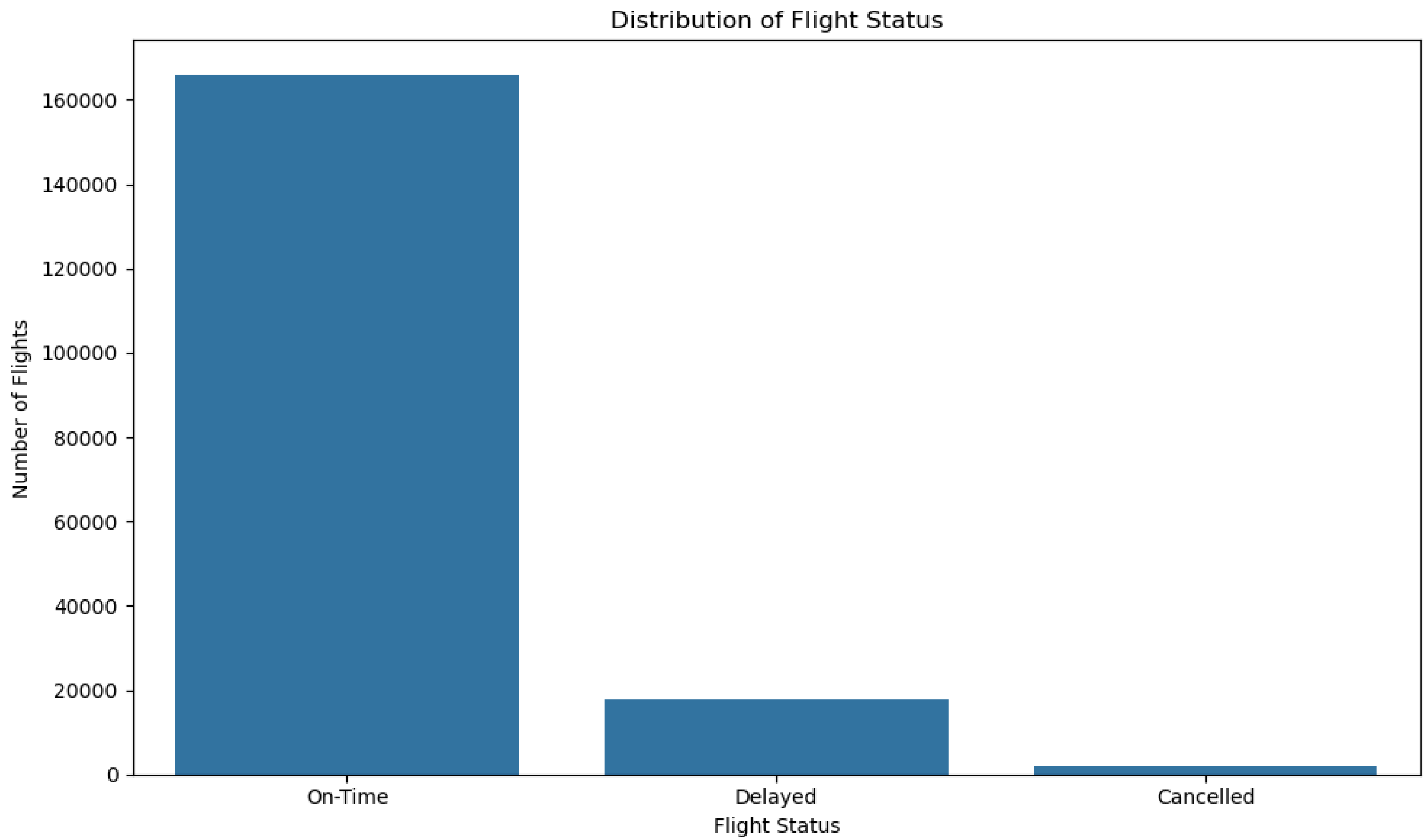
In [16]:
```
"""
Funciton that will categorize Canceled, Delayed, and On-Time flights
"""
def classify_fl_status(row):
    if row['CANCELLED'] == 1:
        return 'Cancelled'
```

```
    elif row['DEP_DEL15'] == 1:
        return 'Delayed'
    elif row['DEP_DEL15'] == 0:
        return 'On-Time'
    else:
        return 'Unknown'
```

In [17]:

```
# create the new column in the dataframe by applying the function
raw_eda_df['Flight_Status'] = raw_eda_df.apply(classify_fl_status, axis=1)

# get the counts of the flight status column
fl_status_counts = raw_eda_df['Flight_Status'].value_counts().reset_index()
# print(fl_status_counts)

# print the percentage of each flight status
fl_status_counts['Percentage'] = (fl_status_counts['count'] / fl_status_counts['count'].sum()) * 100
print(fl_status_counts)

# visualize the distribution of flight status
plt.figure(figsize=(10,6))
sns.countplot(data=raw_eda_df, x='Flight_Status')
plt.title('Distribution of Flight Status')
plt.xlabel('Flight Status')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()
```

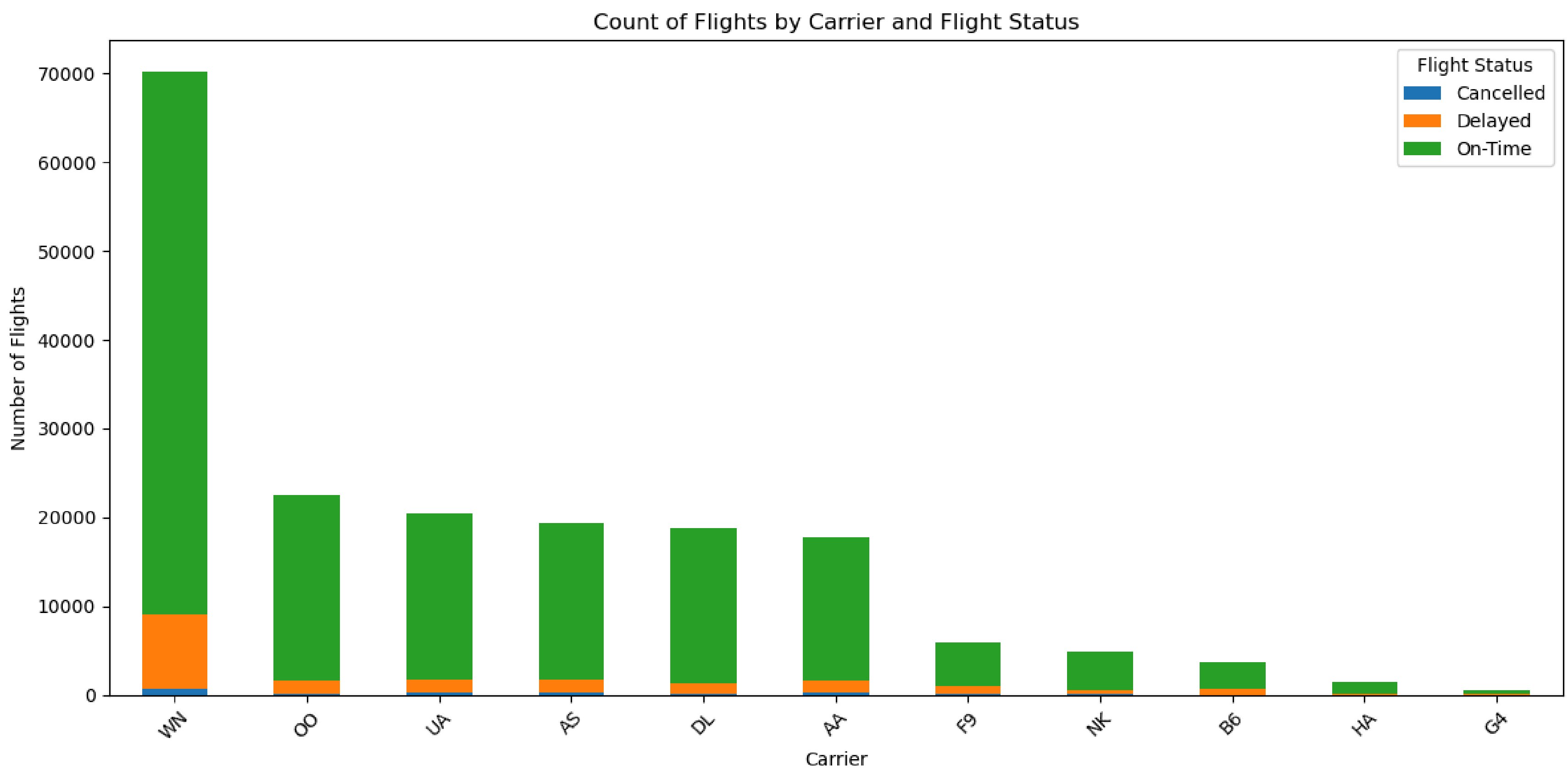
```
  Flight_Status   count  Percentage
0       On-Time  165811   89.337823
1       Delayed   17671    9.521013
2     Cancelled    2118    1.141164
```



In [18]:
```
# explore flight carriers and statuses
# get the count of flights by carrier and status
carrier_status_counts = raw_eda_df.groupby(['OP_UNIQUE_CARRIER', 'Flight_Status']).size().reset_index()
carrier_status_counts.columns = ['Carrier', 'Flight_Status', 'Count']

# pivot table for plotting
carrier_status_pivot = carrier_status_counts.pivot(index='Carrier', columns='Flight_Status', values='Count').fillna(0)
print(carrier_status_pivot)
# sort by total flight count per carrier
carrier_status_pivot['Total'] = carrier_status_pivot.sum(axis=1)
carrier_status_pivot = carrier_status_pivot.sort_values(by='Total', ascending=False)
carrier_status_pivot = carrier_status_pivot.drop(columns='Total')
```

```
# plot stacked bar chart
carrier_status_pivot.plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Count of Flights by Carrier and Flight Status')
plt.xlabel('Carrier')
plt.ylabel('Number of Flights')
plt.legend(title='Flight Status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Flight_Status  Cancelled  Delayed  On-Time
Carrier
AA                   192     1483    16001
AS                   326     1495    17581
B6                    29      634     3060
DL                   141     1211    17482
F9                   133      826     4971
G4                     9      158      425
HA                     8      173     1253
NK                    87      481     4263
OO                   139     1453    20938
UA                   320     1378    18735
WN                   734     8379    61102
```



In [19]:
```
# noticing mismatch in counts before and after
# count how many rows have both cancelled and delayed
both_cancelled_and_delayed = raw_eda_df[
    (raw_eda_df['DEP_DEL15'] == 1) &
    (raw_eda_df['CANCELLED'] == 1)
]

print("Rows where a cancelled flight was also marked as delayed:", len(both_cancelled_and_delayed))

## There are 10 flights that were classified as both cancelled and delayed,
## this accounts for why the counts seem to be off when we examined the counts of the columns previously
## these rows will be classified as cancelled in the processed dataset
```

```
Rows where a cancelled flight was also marked as delayed: 10
```

In [20]:
```
# one hot encode the carrier column
carrier_dummies = pd.get_dummies(raw_eda_df['OP_UNIQUE_CARRIER'], prefix='Carrier')
```

```python
# concatenate the carrier dummies with the original dataframe, we are not going to get rid of the extra column just yet.
raw_eda_df = pd.concat([raw_eda_df, carrier_dummies], axis=1)


print(raw_eda_df.head())
```

```
     FL_DATE CRS_DEP_TIME  DEP_TIME OP_UNIQUE_CARRIER  OP_CARRIER_FL_NUM  \
0  2023-01-01     06:10:00  06:05:00                DL                977
1  2023-01-01     06:15:00  06:17:00                DL               2423
2  2023-01-01     06:15:00  06:44:00                AA               1651
3  2023-01-01     06:15:00  06:15:00                DL                820
4  2023-01-01     06:15:00  06:13:00                DL                914

  DEST  DEP_DEL15  CANCELLED CANCELLATION_CODE  CARRIER_DELAY  ...  \
0  DTW          0          0               NaN              0  ...
1  SLC          0          0               NaN              0  ...
2  CLT          1          0               NaN              0  ...
3  ATL          0          0               NaN              0  ...
4  MSP          0          0               NaN              0  ...

   Carrier_AS  Carrier_B6  Carrier_DL  Carrier_F9  Carrier_G4  Carrier_HA  \
0       False       False        True       False       False       False
1       False       False        True       False       False       False
2       False       False       False       False       False       False
3       False       False        True       False       False       False
4       False       False        True       False       False       False

   Carrier_NK  Carrier_OO  Carrier_UA  Carrier_WN
0       False       False       False       False
1       False       False       False       False
2       False       False       False       False
3       False       False       False       False
4       False       False       False       False

[5 rows x 33 columns]
```

Exploration of Weather Features

In [21]:

```python
# get dtype of weather columns
weather_columns = ['wind_dir_degrees', 'wind_speed_kt', 'visibility_statute_mi', 'temperature_c', 'dewpoint_c', 'altimeter_hpa']
raw_eda_df[weather_columns].dtypes
```

```
Out[21]: wind_dir_degrees         float64
         wind_speed_kt            float64
         visibility_statute_mi    float64
         temperature_c            float64
         dewpoint_c               float64
         altimeter_hpa            float64
         dtype: object
```

```
In [22]: # check unique values for wind direction
         raw_eda_df['wind_dir_degrees'].unique()
```

```
Out[22]: array([290., 280., 270.,   0., 130., 140.,  nan, 210., 240., 260., 220.,
                160., 120., 150., 170., 190., 180., 200., 250., 350., 340., 320.,
                300., 310., 330.,  20.,  90.,  50., 360.,  10.,  40., 230.,  60.,
                 70., 110., 100.,  30.,  80.])
```

```
In [23]: # since 0 and 360 are the same, we will replace 0 with 360
         raw_eda_df['wind_dir_degrees'] = raw_eda_df['wind_dir_degrees'].replace(0.0, 360.0)

         # Address missing values in wind_dir_degrees to change to integer and address missing values
         # Missing values are attributed to what is known as Variable Wind speeds, which are often associated with relatively calm wind sp
         # We will replace these missing values with the mode of the wind_dir_degrees column
         # get the mode of wind_dir_degrees
         mode_wind_dir = raw_eda_df['wind_dir_degrees'].mode()[0]
         print("Mode of wind_dir_degrees:", mode_wind_dir)

         # replace missing values with the mode
         raw_eda_df['wind_dir_degrees'] = raw_eda_df['wind_dir_degrees'].fillna(mode_wind_dir)

         # get the count of missing values in wind_dir_degrees
         missing_wind_dir = raw_eda_df['wind_dir_degrees'].isnull().sum()
         print("Missing values in wind_dir_degrees:", missing_wind_dir)
```

```
Mode of wind_dir_degrees: 360.0
Missing values in wind_dir_degrees: 0
```

```
In [24]: # convert the wind_dir_degrees column to integer
         raw_eda_df['wind_dir_degrees'] = raw_eda_df['wind_dir_degrees'].astype(int)
```

```
In [25]: # explore unique values for wind speed
         print(raw_eda_df['wind_speed_kt'].unique())

         # convert them to integers
         raw_eda_df['wind_speed_kt'] = raw_eda_df['wind_speed_kt'].astype(int)
```

```
[ 7. 11. 17. 15. 13. 16. 20. 14. 19. 18. 12.  9.  8.  0.  3.  5.  6. 10.
  4. 21. 22.]
```

In [26]:
```python
# explore unique values for wind gust
print(raw_eda_df['wind_gust_kt'].unique())

# convert them to integers
raw_eda_df['wind_gust_kt'] = raw_eda_df['wind_gust_kt'].astype(int)
```

```
[19. 25.  0. 24. 23. 28. 26. 21. 18. 20. 22. 16. 17. 27. 15. 30. 34. 14.
 29. 42. 32. 35. 31. 33.]
```

In [27]:
```python
# explore unique values for visibility
print(raw_eda_df['visibility_statute_mi'].unique())
```

```
[ 6.     3.     9.    10.     8.     4.     2.5    1.5    7.     2.
  5.     1.     1.75   1.25   0.75   0.25   0.5    0.125]
```

In [28]:
```python
# check temperature unique values
print(raw_eda_df['temperature_c'].unique())
```

```
[ 13.3  13.9  14.4  15.   15.6  11.7  12.2  12.8  14.   16.1  16.   16.7
  17.8  17.2   9.4  10.   11.1   8.3   7.8  10.6  18.9  19.4  18.3   8.9
  20.6  21.1  22.2  20.   17.   13.    5.6   6.1   4.4   7.2   5.   21.7
   6.7  24.4  23.3  22.8  12.   23.9  25.6  26.7  26.1  27.2  25.   18.
   0.   19.   21.   27.8  27.   22.   23.   28.9  28.3 -17.8  24.   11.
   3.9  29.4  30.   30.6  31.1  31.7  32.2  32.8  34.4]
```

In [29]:
```python
# check dewpoint unique values
print(raw_eda_df['dewpoint_c'].unique())
```

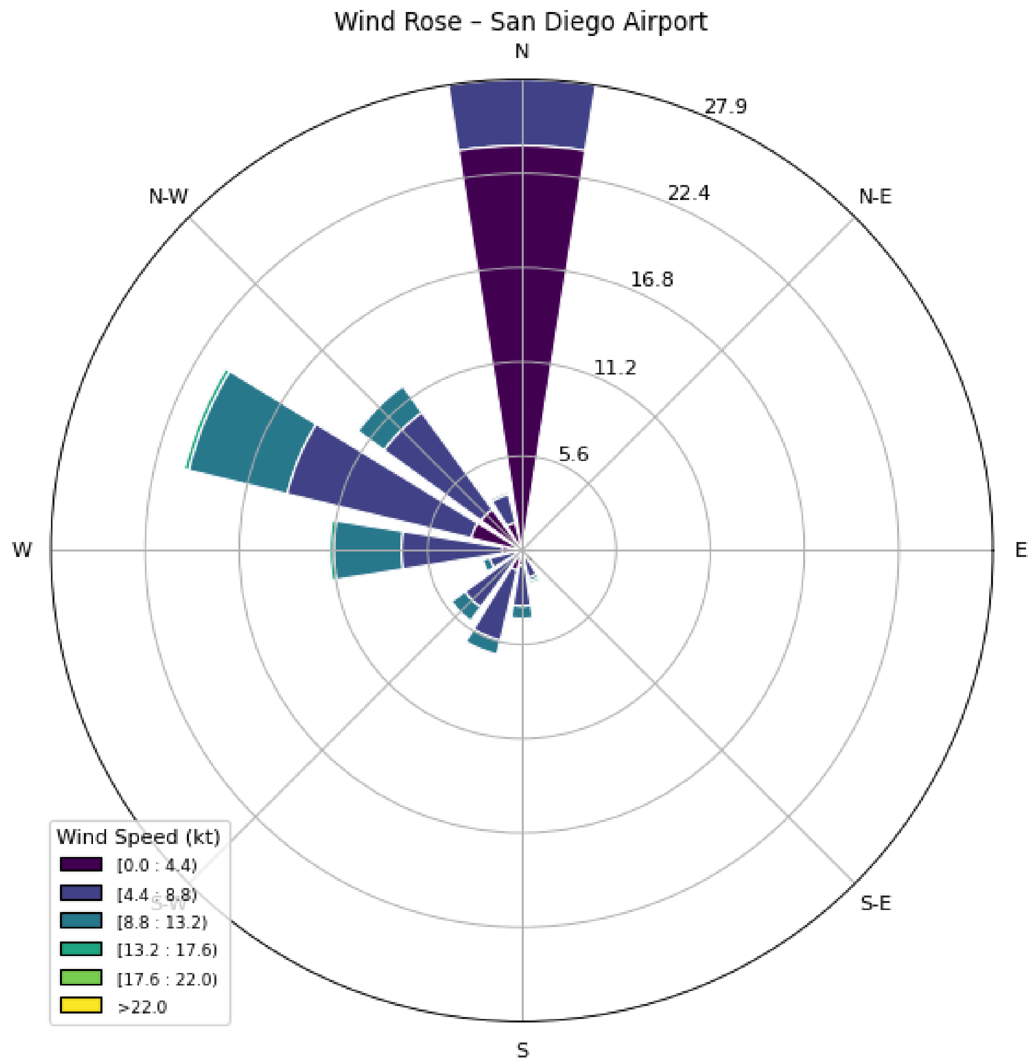
```
[ 10.    9.4   8.3   8.9   7.8  10.6  11.   11.1  11.7  12.   12.2  13.3
  12.8  13.9  14.   13.   14.4   5.    5.6   6.1   3.3   4.4   2.8   0.6
   2.2   3.9   6.7   7.2   1.1   8.   -2.2  -1.7   1.7  -3.9  -3.3  -6.7
  -6.1  -4.4  -0.6   0.   -5.   -7.8  -7.2  -8.3  -5.6  -2.8  -1.1  -8.9
   7.  -10.6 -12.8 -12.2 -11.7 -13.3   9.    6.   15.   15.6  16.   16.1
  16.7  17.2  17.8  18.3  18.9  17.   18.   19.   19.4  21.1  21.7  22.2
  22.8  20.   20.6  21.  -17.8  10.9  22.  -11.1  -9.4]
```

In [30]:
```python
# check altimeter unique values
print(raw_eda_df['altimeter_hpa'].unique())
```

```
[29.82 29.83 29.84 29.86 29.88 29.85 29.9  29.91 29.97 29.98 30.01 30.03
 30.05 30.02 30.04 30.07 30.08 30.09 30.06 30.1  30.11 30.12 30.14 30.15
 30.16 30.17 30.13 30.19 30.21 30.23 30.24 30.25 30.22 30.2  30.18 30.28
 30.   29.99 29.95 29.94 29.93 29.92 29.89 29.96 29.87 29.79 29.8  29.81
 30.26 30.27 30.3  30.31 30.33 30.34 30.29 30.35 30.36 30.32 29.78 29.77
 29.76 29.61 29.58 29.56 29.54 29.51 29.49 29.46 29.45 29.47 29.48 29.52
 29.59 29.63 29.64 29.71 29.72 29.75 29.74 29.73 29.7  29.68 29.67 29.69]
```

In [31]:

```python
# develop a windrose visualization to visualize the most common direction and speeds.
from windrose import WindroseAxes

wind_rose_df = raw_eda_df[['wind_dir_degrees', 'wind_speed_kt']]

ax = WindroseAxes.from_ax()
ax.bar(
    wind_rose_df['wind_dir_degrees'],
    wind_rose_df['wind_speed_kt'],
    normed=True,
    opening=0.8,
    edgecolor='white'
)
ax.set_title('Wind Rose - San Diego Airport')
ax.set_legend(title="Wind Speed (kt)")
plt.show()
```



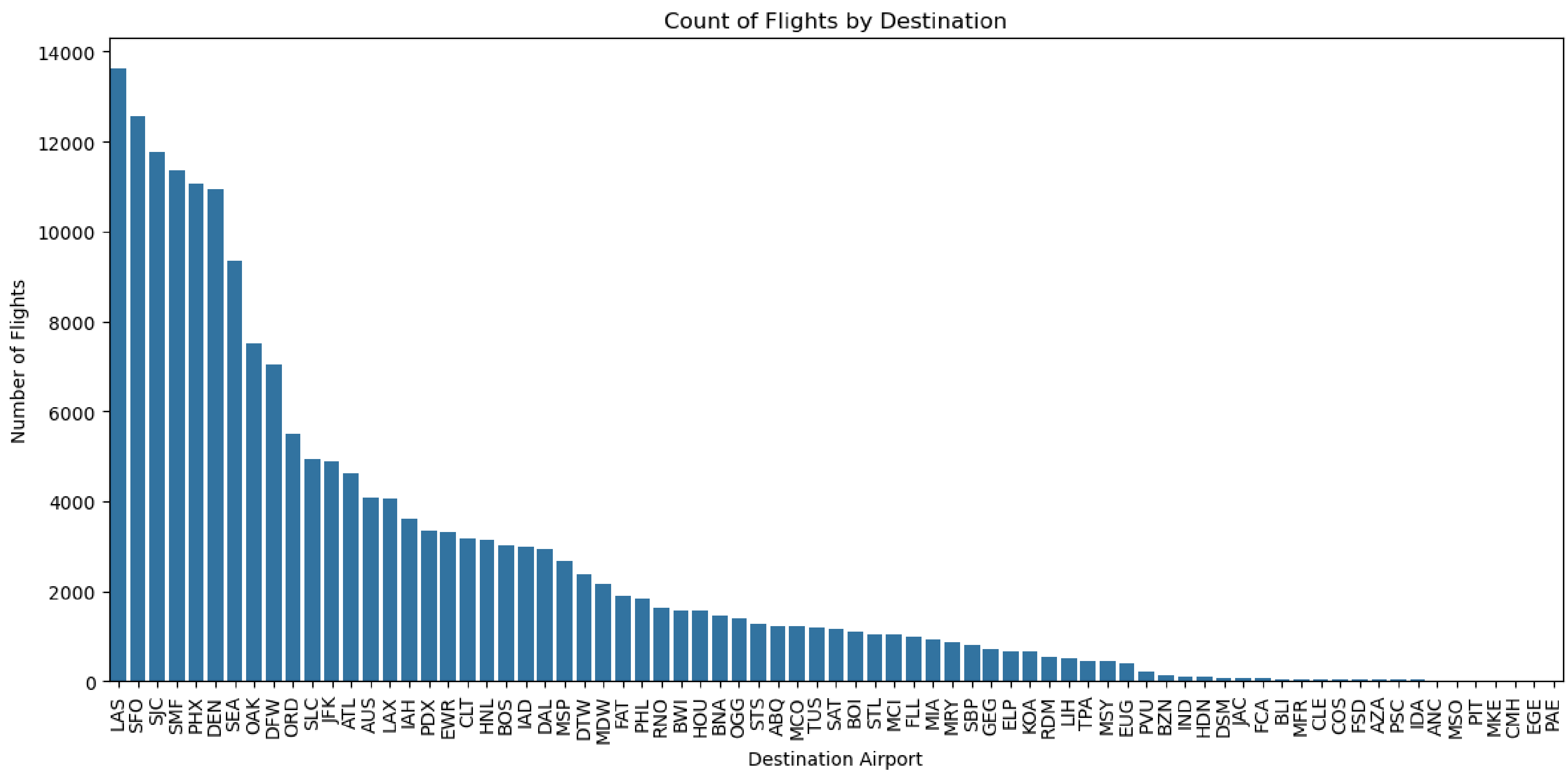
Destination Exploration

In [32]:
```
# check the unique values for DEST
print(raw_eda_df['DEST'].unique())
```

```
['DTW' 'SLC' 'CLT' 'ATL' 'MSP' 'LAS' 'PHX' 'DFW' 'DEN' 'SMF' 'SJC' 'OAK'
 'LAX' 'SFO' 'ORD' 'PHL' 'AUS' 'JFK' 'SEA' 'EWR' 'DAL' 'HNL' 'LIH' 'IAH'
 'MCO' 'BNA' 'IAD' 'OGG' 'BOS' 'RDM' 'MDW' 'ELP' 'RNO' 'HOU' 'HDN' 'KOA'
 'SAT' 'MRY' 'MCI' 'FLL' 'ABQ' 'BOI' 'PDX' 'SBP' 'TUS' 'STS' 'BWI' 'STL'
 'FAT' 'GEG' 'MIA' 'MSY' 'COS' 'DSM' 'PVU' 'BZN' 'JAC' 'EUG' 'FSD' 'BLI'
 'AZA' 'PSC' 'IDA' 'CLE' 'MFR' 'IND' 'FCA' 'MSO' 'TPA' 'PAE' 'ANC' 'PIT'
 'CMH' 'MKE' 'EGE']
```

In [33]:

```python
# explore visualization of flights by destination
plt.figure(figsize=(12, 6))
sns.countplot(data=raw_eda_df, x='DEST', order=raw_eda_df['DEST'].value_counts().index)
plt.title('Count of Flights by Destination')
plt.xlabel('Destination Airport')
plt.ylabel('Number of Flights')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



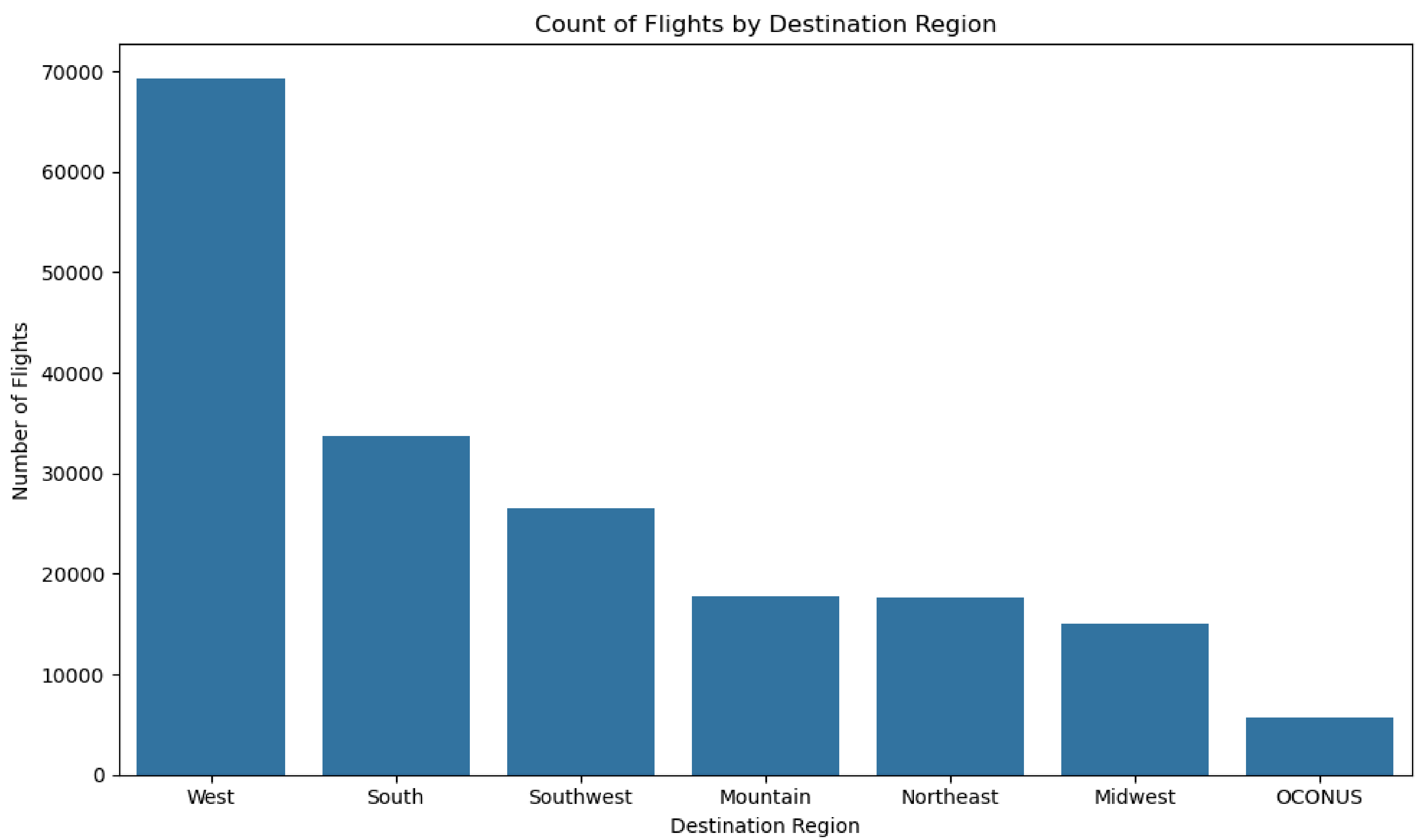
In [34]:

```python
# map airport codes to region
region_map = {
    # West
    'LAX': 'West', 'SFO': 'West', 'OAK': 'West', 'SJC': 'West', 'SMF': 'West',
    'SBP': 'West', 'STS': 'West', 'SAN': 'West', 'FAT': 'West', 'MRY': 'West',
    'BOI': 'West', 'PDX': 'West', 'RDM': 'West', 'EUG': 'West', 'BLI': 'West',
    'PAE': 'West', 'RNO': 'West', 'PSC': 'West', 'SMF': 'West', 'SEA': 'West',
    'GEG': 'West', 'MFR': 'West',

    # Mountain
    'SLC': 'Mountain', 'DEN': 'Mountain', 'COS': 'Mountain', 'HDN': 'Mountain',
    'ABQ': 'Mountain', 'BZN': 'Mountain', 'JAC': 'Mountain', 'IDA': 'Mountain',
    'MSO': 'Mountain', 'FCA': 'Mountain', 'PVU': 'Mountain', 'EGE': 'Mountain',

    # Southwest
    'PHX': 'Southwest', 'TUS': 'Southwest', 'LAS': 'Southwest', 'AZA': 'Southwest',
    'ELP': 'Southwest',

    # Midwest
    'ORD': 'Midwest', 'MDW': 'Midwest', 'MCI': 'Midwest', 'STL': 'Midwest',
    'DTW': 'Midwest', 'MSP': 'Midwest', 'IND': 'Midwest', 'CMH': 'Midwest',
    'CLE': 'Midwest', 'MKE': 'Midwest', 'DSM': 'Midwest', 'FSD': 'Midwest',

    # South
    'CLT': 'South', 'ATL': 'South', 'BNA': 'South', 'DFW': 'South', 'DAL': 'South', 'AUS': 'South',
    'IAH': 'South', 'HOU': 'South', 'SAT': 'South', 'MSY': 'South', 'MCO': 'South',
    'TPA': 'South', 'FLL': 'South', 'MIA': 'South',

    # Northeast
    'BOS': 'Northeast', 'PHL': 'Northeast', 'JFK': 'Northeast', 'EWR': 'Northeast',
    'PIT': 'Northeast', 'IAD': 'Northeast', 'BWI': 'Northeast',

    # OCONUS will serve as the placeholder for Alaska and Hawaii flights
    # OCONUS refers to Outside Continental United States
    'ANC': 'OCONUS', 'HNL': 'OCONUS', 'KOA': 'OCONUS', 'OGG': 'OCONUS', 'LIH': 'OCONUS',
}
# DEST to region
raw_eda_df['DEST_REGION'] = raw_eda_df['DEST'].map(region_map).fillna('Other')

# results
print(raw_eda_df[['DEST', 'DEST_REGION']].head())
```

```
  DEST DEST_REGION
0  DTW     Midwest
1  SLC    Mountain
2  CLT       South
3  ATL       South
4  MSP     Midwest
```

In [35]:

```python
# visualize the distribution of flights by region
plt.figure(figsize=(10,6))
sns.countplot(data=raw_eda_df, x='DEST_REGION', order=raw_eda_df['DEST_REGION'].value_counts().index)
plt.title('Count of Flights by Destination Region')
plt.xlabel('Destination Region')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()

# print counts of flights by region
print(raw_eda_df['DEST_REGION'].value_counts())
```

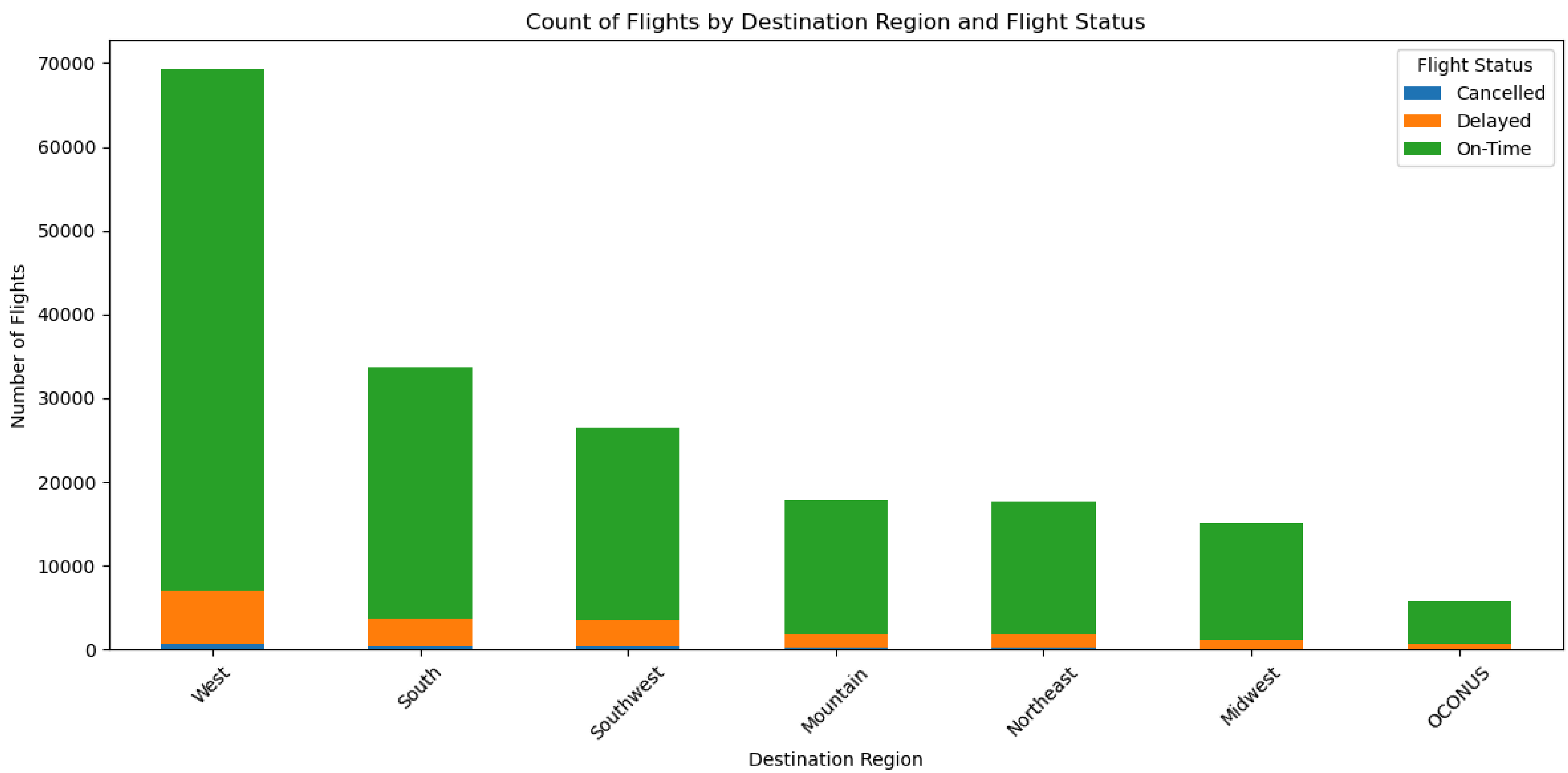



```
DEST_REGION
West         69248
South        33648
Southwest    26551
Mountain     17746
Northeast    17653
Midwest      15044
OCONUS        5710
Name: count, dtype: int64
```

In [36]:
```
# creating a stacked bar chart of destination region and flight status
region_status_counts = raw_eda_df.groupby(['DEST_REGION', 'Flight_Status']).size().reset_index()
region_status_counts.columns = ['Region', 'Flight_Status', 'Count']
```

```python
# pivot table for plotting
region_status_pivot = region_status_counts.pivot(index='Region', columns='Flight_Status', values='Count').fillna(0)
print(region_status_pivot)

# sort by total flight count per region
region_status_pivot['Total'] = region_status_pivot.sum(axis=1)
region_status_pivot = region_status_pivot.sort_values(by='Total', ascending=False)
region_status_pivot = region_status_pivot.drop(columns='Total')

# plot stacked bar chart
region_status_pivot.plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Count of Flights by Destination Region and Flight Status')
plt.xlabel('Destination Region')
plt.ylabel('Number of Flights')
plt.legend(title='Flight Status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Flight_Status  Cancelled  Delayed  On-Time
Region
Midwest              128     1115    13801
Mountain             198     1658    15890
Northeast            223     1502    15928
OCONUS                56      609     5045
South                443     3315    29890
Southwest            387     3080    23084
West                 683     6392    62173
```



Date Exploration

In [ ]:

In [37]:

```
# One hot encode the DEST_REGION column
dest_region_dummies = pd.get_dummies(raw_eda_df['DEST_REGION'], prefix='Dest_Region')

# concatenate the carrier dummies with the original dataframe, we are not going to get rid of the extra column just yet.
raw_eda_df = pd.concat([raw_eda_df, dest_region_dummies], axis=1)
```

In [38]:

```
print(raw_eda_df.dtypes)
```

```
FL_DATE                     object
CRS_DEP_TIME                object
DEP_TIME                    object
OP_UNIQUE_CARRIER           object
OP_CARRIER_FL_NUM            int64
DEST                        object
DEP_DEL15                    int64
CANCELLED                    int64
CANCELLATION_CODE           object
CARRIER_DELAY                int64
WEATHER_DELAY                int64
NAS_DELAY                    int64
SECURITY_DELAY               int64
LATE_AIRCRAFT_DELAY          int64
wind_dir_degrees             int64
wind_speed_kt                int64
wind_gust_kt                 int64
visibility_statute_mi      float64
temperature_c              float64
dewpoint_c                 float64
altimeter_hpa              float64
Flight_Status               object
Carrier_AA                    bool
Carrier_AS                    bool
Carrier_B6                    bool
Carrier_DL                    bool
Carrier_F9                    bool
Carrier_G4                    bool
Carrier_HA                    bool
Carrier_NK                    bool
Carrier_OO                    bool
Carrier_UA                    bool
Carrier_WN                    bool
DEST_REGION                 object
Dest_Region_Midwest           bool
Dest_Region_Mountain          bool
Dest_Region_Northeast         bool
Dest_Region_OCONUS            bool
Dest_Region_South             bool
Dest_Region_Southwest         bool
Dest_Region_West              bool
dtype: object
```

Finalize data frame for processed data

In [39]:

```
processed_df = raw_eda_df.copy()
processed_df = processed_df.drop(columns=['DEP_TIME', 'DEP_DEL15', 'CANCELLED', 'OP_CARRIER_FL_NUM', 'CANCELLATION_CODE', 'CARRI

# create the target variable for the model using Flight_Status where if Cancelled or Delayed = 1 and On-time = 0
processed_df['Flight_Status_Binary'] = processed_df['Flight_Status'].apply(
    lambda x: 1 if x in ['Delayed', 'Cancelled'] else 0
)

#print head
print(processed_df.head())
```

```
     FL_DATE CRS_DEP_TIME OP_UNIQUE_CARRIER DEST  wind_dir_degrees  \
0  2023-01-01     06:10:00                DL  DTW               290
1  2023-01-01     06:15:00                DL  SLC               290
2  2023-01-01     06:15:00                AA  CLT               290
3  2023-01-01     06:15:00                DL  ATL               290
4  2023-01-01     06:15:00                DL  MSP               290

   wind_speed_kt  wind_gust_kt  visibility_statute_mi  temperature_c  \
0              7            19                    6.0           13.3
1              7            19                    6.0           13.3
2              7            19                    6.0           13.3
3              7            19                    6.0           13.3
4              7            19                    6.0           13.3

   dewpoint_c  ...  Carrier_WN DEST_REGION  Dest_Region_Midwest  \
0        10.0  ...       False     Midwest                 True
1        10.0  ...       False    Mountain                False
2        10.0  ...       False       South                False
3        10.0  ...       False       South                False
4        10.0  ...       False     Midwest                 True

   Dest_Region_Mountain  Dest_Region_Northeast  Dest_Region_OCONUS  \
0                 False                  False               False
1                  True                  False               False
2                 False                  False               False
3                 False                  False               False
4                 False                  False               False

   Dest_Region_South  Dest_Region_Southwest  Dest_Region_West  \
0              False                  False             False
1              False                  False             False
2               True                  False             False
3               True                  False             False
4              False                  False             False

   Flight_Status_Binary
0                     0
1                     0
2                     1
3                     0
4                     0

[5 rows x 32 columns]
```

In [40]:

```
# save the processed data to a csv file
```

```
processed_df.to_csv("processed_data/processed_data.csv", index=False)
```

Upload processed_data.csv to Processed_data S3 Bucket

In [41]:
```
# examine list of buckets available
s3 = boto3.client('s3')
response = s3.list_buckets()
print("Existing buckets:")
for bucket in response['Buckets']:
    print(bucket['Name'])
```

```
Existing buckets:
group9-ml-proj-athena-query-bucket-grw
group9-ml-proj-model-artifacts-bucket-grw
group9-ml-proj-monitoring-bucket-grw
group9-ml-proj-processed-data-bucket-grw
group9-ml-proj-raw-data-bucket-grw
group9-ml-proj-training-artifacts-bucket-grw
sagemaker-studio-7sb8gxpljq9
sagemaker-studio-chxawrkbs1q
sagemaker-studio-o4i4ukj5y3b
sagemaker-us-east-1-804823187130
```

In [42]:
```
# set local path of organized weather CSV file
local_file_path = 'processed_data/processed_data.csv'
```

In [43]:
```
# set the raw data bucket variable
processed_bucket = "group9-ml-proj-processed-data-bucket-grw"
```

In [44]:
```
# set the S3 key path, there will be
s3_key = "all_data/processed_data.csv"
```

In [45]:
```
# upload file to the S3 bucket
try:
    s3.upload_file(local_file_path, processed_bucket, s3_key)
    print(f"Uploaded {local_file_path} to s3://{processed_bucket}/{s3_key}")
except Exception as e:
    print("Error uploading file to S3:", e)
```

```
Uploaded processed_data/processed_data.csv to s3://group9-ml-proj-processed-data-bucket-grw/all_data/processed_data.csv
```

In [46]:
```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button
```

```
<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.

```
In [ ]:
```

In [1]:
```
!pip install imblearn
```

```
Collecting imblearn
  Downloading imblearn-0.0-py2.py3-none-any.whl.metadata (355 bytes)
Collecting imbalanced-learn (from imblearn)
  Downloading imbalanced_learn-0.13.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: numpy<3,>=1.24.3 in /opt/conda/lib/python3.11/site-packages (from imbalanced-learn->imblearn) (1.2
6.4)
Requirement already satisfied: scipy<2,>=1.10.1 in /opt/conda/lib/python3.11/site-packages (from imbalanced-learn->imblearn) (1.1
5.2)
Requirement already satisfied: scikit-learn<2,>=1.3.2 in /opt/conda/lib/python3.11/site-packages (from imbalanced-learn->imblear
n) (1.5.2)
Collecting sklearn-compat<1,>=0.1 (from imbalanced-learn->imblearn)
  Downloading sklearn_compat-0.1.3-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: joblib<2,>=1.1.1 in /opt/conda/lib/python3.11/site-packages (from imbalanced-learn->imblearn) (1.
4.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /opt/conda/lib/python3.11/site-packages (from imbalanced-learn->imblear
n) (3.5.0)
Downloading imblearn-0.0-py2.py3-none-any.whl (1.9 kB)
Downloading imbalanced_learn-0.13.0-py3-none-any.whl (238 kB)
Downloading sklearn_compat-0.1.3-py3-none-any.whl (18 kB)
Installing collected packages: sklearn-compat, imbalanced-learn, imblearn
Successfully installed imbalanced-learn-0.13.0 imblearn-0.0 sklearn-compat-0.1.3
```

In [1]:
```
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
import boto3
import time
import io
```

In [2]:
```
# Athena Query of the S3 bucket function
def run_athena_query(query, database, output_location):
    athena_client = boto3.client('athena')

    # start the query execution.
    response = athena_client.start_query_execution(
        QueryString=query,
        QueryExecutionContext={'Database': database},
        ResultConfiguration={'OutputLocation': output_location}
    )
    query_execution_id = response['QueryExecutionId']
    print(f"Query submitted. Execution ID: {query_execution_id}")
```

```python
    # poll the query status until it completes.
    state = 'RUNNING'
    while state in ['RUNNING', 'QUEUED']:
        response = athena_client.get_query_execution(QueryExecutionId=query_execution_id)
        state = response['QueryExecution']['Status']['State']
        print(f"Current query state: {state}")
        if state in ['RUNNING', 'QUEUED']:
            time.sleep(2)

    if state == 'FAILED':
        reason = response['QueryExecution']['Status'].get('StateChangeReason', 'Unknown error')
        raise Exception(f"Query {query_execution_id} failed: {reason}")

    print(f"Query {query_execution_id} completed successfully.")
    return query_execution_id
```

In [3]:

```python
# turn query results into DF function
def get_query_results_as_dataframe(query_execution_id):
    athena_client = boto3.client('athena')
    s3_client = boto3.client('s3')

    # get query execution to find the S3 location of the results.
    response = athena_client.get_query_execution(QueryExecutionId=query_execution_id)
    result_location = response['QueryExecution']['ResultConfiguration']['OutputLocation']

    # parse S3 bucket and key from the result loc.
    s3_path = result_location.replace("s3://", "").split("/", 1)
    bucket = s3_path[0]
    key = s3_path[1]

    # get the CSV result file from S3.
    obj = s3_client.get_object(Bucket=bucket, Key=key)
    df = pd.read_csv(io.BytesIO(obj['Body'].read()))
    return df
```

In [5]:

```python
# configuration and Query Execution
# set Athena db name and output location.
athena_database = 'processed_data'
output_location = 's3://group9-ml-proj-athena-query-bucket-grw/'

# define your query.
query = """
SELECT wind_dir_degrees,
       wind_speed_kt,
       wind_gust_kt,
```

```
        visibility_statute_mi,
        temperature_c,
        dewpoint_c,
        altimeter_hpa,
        Carrier_AA,
        Carrier_AS,
        Carrier_B6,
        Carrier_DL,
        Carrier_F9,
        Carrier_G4,
        Carrier_HA,
        Carrier_NK,
        Carrier_OO,
        Carrier_UA,
        Carrier_WN,
        Dest_Region_Midwest,
        Dest_Region_Mountain,
        Dest_Region_Northeast,
        Dest_Region_OCONUS,
        Dest_Region_South,
        Dest_Region_Southwest,
        Dest_Region_West,
        Flight_Status_Binary
FROM processed_data;
"""

# run the Athena query and return execution ID.
query_execution_id = run_athena_query(query, database=athena_database, output_location=output_location)

# retrieve the query results as a pandas df named 'data'
data = get_query_results_as_dataframe(query_execution_id)

# display the first few rows of the data
print(data.head())
```

```
Query submitted. Execution ID: 060667fd-9695-49e3-bba8-d822127a50a0
Current query state: QUEUED
Current query state: RUNNING
Current query state: SUCCEEDED
Query 060667fd-9695-49e3-bba8-d822127a50a0 completed successfully.
   wind_dir_degrees  wind_speed_kt  wind_gust_kt  visibility_statute_mi  \
0             290.0            7.0          19.0                    6.0   
1             290.0            7.0          19.0                    6.0   
2             290.0            7.0          19.0                    6.0   
3             290.0            7.0          19.0                    6.0   
4             290.0            7.0          19.0                    6.0   

   temperature_c  dewpoint_c  altimeter_hpa  Carrier_AA  Carrier_AS  \
0           13.3        10.0          29.82       False       False   
1           13.3        10.0          29.82       False       False   
2           13.3        10.0          29.82        True       False   
3           13.3        10.0          29.82       False       False   
4           13.3        10.0          29.82       False       False   

   Carrier_B6  ...  Carrier_UA  Carrier_WN  Dest_Region_Midwest  \
0       False  ...       False       False                 True   
1       False  ...       False       False                False   
2       False  ...       False       False                False   
3       False  ...       False       False                False   
4       False  ...       False       False                 True   

   Dest_Region_Mountain  Dest_Region_Northeast  Dest_Region_OCONUS  \
0                 False                  False               False   
1                  True                  False               False   
2                 False                  False               False   
3                 False                  False               False   
4                 False                  False               False   

   Dest_Region_South  Dest_Region_Southwest  Dest_Region_West  \
0              False                  False             False   
1              False                  False             False   
2               True                  False             False   
3               True                  False             False   
4              False                  False             False   

   Flight_Status_Binary  
0                 False  
1                 False  
2                 False  
3                 False  
4                 False  
```

[5 rows x 26 columns]

Split 70/15/15

Undersample

In [4]:

```
from sklearn.model_selection import train_test_split
from imblearn.under_sampling import RandomUnderSampler

# Define features (X) and target (y)
X = data.drop(['Flight_Status_Binary'], axis=1)
y = data['Flight_Status_Binary']

# Step 1: Split into training (70%) and temp dataset (30%)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)

# Step 2: Further split temp into validation (15%) and testing (15%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=42
)

# Step 3: Perform undersampling on the training dataset only
rus = RandomUnderSampler(random_state=42)
X_train_resampled, y_train_resampled = rus.fit_resample(X_train, y_train)

# Verify the shape and class distributions
print("Dataset Shapes:")
print(f"Original Training Set: {X_train.shape}")
print(f"Resampled Training Set: {X_train_resampled.shape}")
print(f"Validation Set: {X_val.shape}")
print(f"Testing Set: {X_test.shape}")

print("\nTraining Set Class Distribution (Original):")
print(y_train.value_counts())

print("\nTraining Set Class Distribution (After Resampling):")
print(y_train_resampled.value_counts())
```

```
Dataset Shapes:
Original Training Set: (129920, 25)
Resampled Training Set: (27704, 25)
Validation Set: (27840, 25)
Testing Set: (27840, 25)

Training Set Class Distribution (Original):
Flight_Status_Binary
0    116068
1     13852
Name: count, dtype: int64

Training Set Class Distribution (After Resampling):
Flight_Status_Binary
0    13852
1    13852
Name: count, dtype: int64
```

Modeling Building

Logistic Model

In [5]:

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# Instantiate the model
model = LogisticRegression(max_iter=1000, random_state=42)

# Train model on resampled training data
model.fit(X_train_resampled, y_train_resampled)

# Evaluate on validation data
y_pred_val = model.predict(X_val)

# Performance evaluation
print(classification_report(y_val, y_pred_val))
print(confusion_matrix(y_val, y_pred_val))
```

```
              precision    recall  f1-score   support

           0       0.92      0.57      0.70     24871
           1       0.14      0.59      0.23      2969

    accuracy                           0.57     27840
   macro avg       0.53      0.58      0.46     27840
weighted avg       0.84      0.57      0.65     27840

[[14123 10748]
 [ 1217  1752]]
```

Random Forest

In [6]:
```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_resampled, y_train_resampled)

y_val_pred_rf = rf.predict(X_val)
print("Random Forest Validation Report:")
print(classification_report(y_val, y_val_pred_rf))
```

```
Random Forest Validation Report:
              precision    recall  f1-score   support

           0       0.94      0.64      0.76     24871
           1       0.18      0.66      0.28      2969

    accuracy                           0.65     27840
   macro avg       0.56      0.65      0.52     27840
weighted avg       0.86      0.65      0.71     27840
```

Xgboost

In [7]:
```
from xgboost import XGBClassifier

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb.fit(X_train_resampled, y_train_resampled)

y_val_pred_xgb = xgb.predict(X_val)
```

```
print("XGBoost Validation Report:")
print(classification_report(y_val, y_val_pred_xgb))
```

```
XGBoost Validation Report:
              precision    recall  f1-score   support

           0       0.95      0.66      0.78     24871
           1       0.19      0.69      0.30      2969

    accuracy                           0.66     27840
   macro avg       0.57      0.67      0.54     27840
weighted avg       0.87      0.66      0.73     27840
```

Since our goal number 2 is to reduce delay times by informing logistics, its better to catch more delays. Using a lower threshold to increase recall, since missing a delay would be worse than predicting one that doesn't end up happening

Find which is the best threshold with AUC

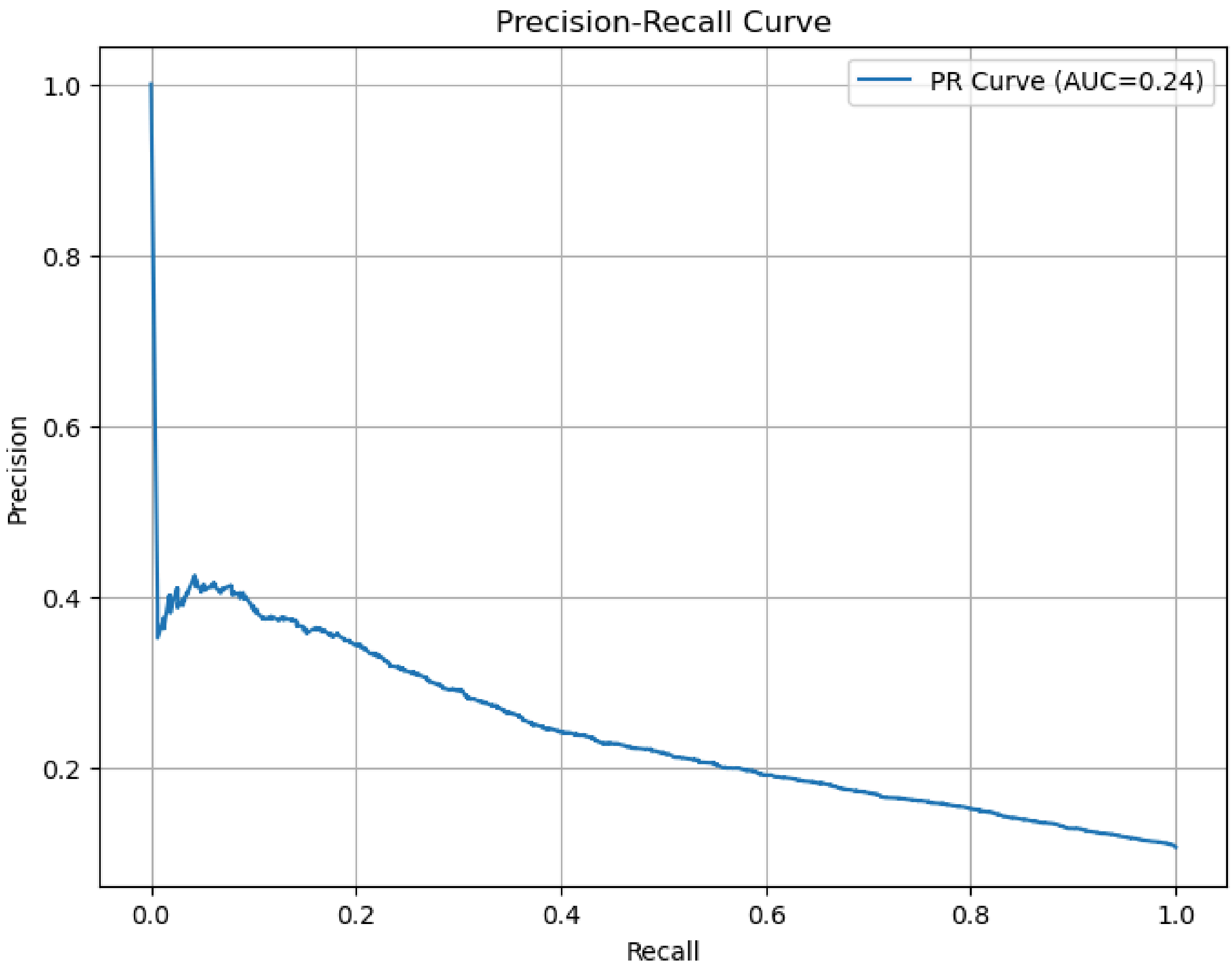
In [8]:
```
from sklearn.ensemble import RandomForestClassifier

# Train a simple random forest model on data
best_rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
best_rf_model.fit(X_train_resampled, y_train_resampled)
```

Out[8]:
```
▾ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

In [9]:
```
# Predict probabilities for the positive class (delays/cancellations)
y_proba_rf = best_rf_model.predict_proba(X_test)[:, 1]
```

In [10]:
```
from sklearn.metrics import precision_recall_curve, auc
import matplotlib.pyplot as plt

precision, recall, thresholds = precision_recall_curve(y_test, y_proba_rf)
pr_auc = auc(recall, precision)

plt.figure(figsize=(8,6))
plt.plot(recall, precision, label=f'PR Curve (AUC={pr_auc:.2f})')
```

```
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend()
plt.grid(True)
plt.show()
```



In [11]:

```
import numpy as np
import pandas as pd
from sklearn.metrics import precision_score, recall_score, f1_score

thresholds = np.arange(0.1, 0.51, 0.05)
results = []
```

```python
for threshold in thresholds:
    y_pred_threshold = (y_proba_rf >= threshold).astype(int)
    precision = precision_score(y_test, y_pred_threshold)
    recall = recall_score(y_test, y_pred_threshold)
    f1 = f1_score(y_test, y_pred_threshold)
    results.append([threshold, precision, recall, f1])

# Convert results to DataFrame
results_df = pd.DataFrame(results, columns=['Threshold', 'Precision', 'Recall', 'F1-score'])
print(results_df)
```

```
   Threshold  Precision    Recall  F1-score
0       0.10   0.111495  0.990229  0.200423
1       0.15   0.114386  0.969003  0.204617
2       0.20   0.118907  0.950135  0.211363
3       0.25   0.124584  0.920148  0.219454
4       0.30   0.133171  0.884771  0.231498
5       0.35   0.142210  0.835916  0.243068
6       0.40   0.154194  0.792790  0.258174
7       0.45   0.164611  0.719340  0.267913
8       0.50   0.181361  0.655660  0.284129
```

In [12]:

```python
final_threshold = 0.25
y_pred_final = (y_proba_rf >= final_threshold).astype(int)

print("Final Threshold (0.25) Classification Report:")
print(classification_report(y_test, y_pred_final))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_final))
```

```
Final Threshold (0.25) Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.22      0.36     24872
           1       0.12      0.92      0.22      2968

    accuracy                           0.30     27840
   macro avg       0.54      0.57      0.29     27840
weighted avg       0.87      0.30      0.35     27840

Confusion Matrix:
[[ 5511 19361]
 [  226  2742]]
```

Random Forest with 0.25

In [13]:
```
# Predict probabilities
y_proba_rf = best_rf_model.predict_proba(X_test)[:, 1]

# Set optimal threshold
threshold = 0.25
y_pred_rf = (y_proba_rf >= threshold).astype(int)

# Evaluation metrics
print("Random Forest with threshold = 0.25")
print(classification_report(y_test, y_pred_rf))
print(confusion_matrix(y_test, y_pred_rf))
```

```
Random Forest with threshold = 0.25
              precision    recall  f1-score   support

           0       0.96      0.22      0.36     24872
           1       0.12      0.92      0.22      2968

    accuracy                           0.30     27840
   macro avg       0.54      0.57      0.29     27840
weighted avg       0.87      0.30      0.35     27840

[[ 5511 19361]
 [  226  2742]]
```

Logistic with 0.25

In [14]:
```
from sklearn.linear_model import LogisticRegression

# Train Logistic Regression on resampled training data
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_resampled, y_train_resampled)

# Predict probabilities
y_proba_lr = lr_model.predict_proba(X_test)[:, 1]

# Apply optimal threshold
y_pred_lr = (y_proba_lr >= threshold).astype(int)

# Print
```

```
print("Logistic Regression with threshold = 0.25")
print(classification_report(y_test, y_pred_lr))
print(confusion_matrix(y_test, y_pred_lr))
```

```
Logistic Regression with threshold = 0.25
              precision    recall  f1-score   support

           0       0.97      0.01      0.02     24872
           1       0.11      1.00      0.19      2968

    accuracy                           0.11     27840
   macro avg       0.54      0.50      0.11     27840
weighted avg       0.88      0.11      0.04     27840

[[  213 24659]
 [    6  2962]]
```

XGBoost with 0.25 threshold

In [15]:

```
from xgboost import XGBClassifier

# Train XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train_resampled, y_train_resampled)

# Predict probabilities
y_proba_xgb = xgb_model.predict_proba(X_test)[:, 1]

# Apply optimal threshold
y_pred_xgb = (y_proba_xgb >= threshold).astype(int)

# Evaluation metrics
print("XGBoost with threshold = 0.25")
print(classification_report(y_test, y_pred_xgb))
print(confusion_matrix(y_test, y_pred_xgb))
```

```
XGBoost with threshold = 0.25
              precision    recall  f1-score   support

           0       0.97      0.23      0.37     24872
           1       0.13      0.95      0.23      2968

    accuracy                           0.31     27840
   macro avg       0.55      0.59      0.30     27840
weighted avg       0.88      0.31      0.36     27840

[[ 5739 19133]
 [  158  2810]]
```

Best model:

After evaluating all three models, Random Forest is the best option for our goal. It catches the most delays (92% recall), which is crucial because missing a delay costs more than occasionally predicting a false alarm. Even though precision is lower, this trade-off makes sense for our business scenario. Let's move forward with the Random Forest model at a 0.25 threshold to best support logistics planning and minimize disruptions.

Random Forest (Hyperparameters):

In [16]:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

# Hyperparameter grid for tuning
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10],
    'class_weight': [None, 'balanced']
}

# Randomized search for best hyperparameters
rf_tuned = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_grid,
    scoring='recall',  # prioritize recall
    cv=5,
    n_iter=5,
    random_state=42,
    verbose=1
)
```

```python
# Fit the tuned Random Forest model
rf_tuned.fit(X_train_resampled, y_train_resampled)

# Check best hyperparameters
print("Best Hyperparameters:", rf_tuned.best_params_)
```

```
Fitting 5 folds for each of 5 candidates, totalling 25 fits
Best Hyperparameters: {'n_estimators': 100, 'min_samples_split': 5, 'max_depth': 20, 'class_weight': None}
```

In [17]:

```python
# Final best parameters
final_rf_model = rf_tuned.best_estimator_

# Predict probabilities on test data
y_proba_final_rf = final_rf_model.predict_proba(X_test)[:, 1]

# Apply threshold (0.25)
final_threshold = 0.25
y_pred_final_rf = (y_proba_final_rf >= final_threshold).astype(int)

# Final evaluation
print("Final Random Forest Model (Tuned Hyperparameters, Threshold = 0.25):")
print(classification_report(y_test, y_pred_final_rf))
print(confusion_matrix(y_test, y_pred_final_rf))
```

```
Final Random Forest Model (Tuned Hyperparameters, Threshold = 0.25):
              precision    recall  f1-score   support

           0       0.98      0.11      0.20     24872
           1       0.12      0.98      0.21      2968

    accuracy                           0.21     27840
   macro avg       0.55      0.54      0.21     27840
weighted avg       0.88      0.21      0.20     27840

[[ 2825 22047]
 [   71  2897]]
```

Conclusion:

After tuning the hyperparameters for our Random Forest model and applying our optimal threshold (0.25), we achieved a good performance with 98% of actual delays. Although this high recall resulted in some false alarms (low precision of 12%), this trade-off is acceptable since missing real delays would cost our logistics operation far more than a few unnecessary alerts.

Training model to cloud with Sagemaker

1. Converted local scikit-learn model into a SageMaker training script (`train.py`)
2. Created `train.csv` and `validation.csv` from our 70/15/15 split
3. Uploaded the datasets to S3 for SageMaker to access
4. Added missing dependency `imblearn` via a `requirements.txt` file
5. Used `sagemaker.sklearn.estimator.SKLearn` to launch the job in the cloud
6. Model trained successfully using a managed `ml.m5.xlarge` instance
7. Model output automatically saved to S3 — fully cloud-compliant

In [18]:
```
# Save training and validation sets to CSV
train_df = X_train.copy()
train_df["Flight_Status_Binary"] = y_train
train_df.to_csv("train.csv", index=False)

val_df = X_val.copy()
val_df["Flight_Status_Binary"] = y_val
val_df.to_csv("validation.csv", index=False)
```

In [19]:
```
train_input = session.upload_data("Model_data/train.csv", key_prefix="ads-508/train")
val_input = session.upload_data("Model_data/validation.csv", key_prefix="ads-508/validation")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[19], line 1
----> 1 train_input = session.upload_data("Model_data/train.csv", key_prefix="ads-508/train")
      2 val_input = session.upload_data("Model_data/validation.csv", key_prefix="ads-508/validation")

NameError: name 'session' is not defined
```

In [ ]:
```
import sagemaker
from sagemaker import get_execution_role
from sagemaker.sklearn.estimator import SKLearn

# Set up session and role
session = sagemaker.Session()
role = get_execution_role()

# Define the estimator
sklearn_estimator = SKLearn(
```

```
    entry_point="Model_data/train.py",
    role=role,
    instance_type= "ml.m5.large",
    framework_version="1.0-1",
    py_version="py3",
    sagemaker_session=session,
    dependencies=["Model_data/requirements.txt"]
)

# Launch SageMaker training job
sklearn_estimator.fit({
    "train": train_input,
    "validation": val_input
})
```

In [ ]:
```
from sagemaker.sklearn.estimator import SKLearn

sklearn_estimator = SKLearn.attach("sagemaker-scikit-learn-2025-03-29-21-43-12-337")
```

In [ ]:
```
predictor = sklearn_estimator.deploy(
    initial_instance_count=1,
    instance_type="ml.m5.large"
)
```

Comments for Professor:

Errors Encountered During Development-->

ResourceLimitExceeded:

The requested resource ml.t3.medium is not available in this region This occurred when trying to launch a training job using the ml.t3.medium instance type. The instance was not available in the us-east-1 region, leading to a resource limit error.

AccessDeniedException:

Not authorized to perform: sagemaker: CreateModel After switching to an available instance type (ml.m5.xlarge and ml.m5.large), an error occurred when attempting to deploy the trained model as an inference endpoint. This was due to an IAM policy restriction that explicitly denied the sagemaker:CreateModel action. Since deployment was not required for this phase of the assignment, the project proceeded without creating an endpoint.

Final Model built-in SageMaker XGBoost Estimator

In [21]:
```python
import sagemaker
from sagemaker import get_execution_role

# Set up SageMaker session and role
session = sagemaker.Session()
role = get_execution_role()
```

In [23]:
```python
# Upload data to S3
train_input = session.upload_data("Model_data/train.csv", key_prefix="ads-508/train")
val_input = session.upload_data("Model_data/validation.csv", key_prefix="ads-508/validation")
```

In [25]:
```python
from sagemaker.inputs import TrainingInput
from sagemaker.xgboost.estimator import XGBoost

xgb_estimator = XGBoost(
    entry_point="Model_data/xgboost_train.py",
    role=role,
    instance_type="ml.m5.large",
    instance_count=1,
    framework_version="1.3-1",
    py_version="py3",
    sagemaker_session=session
)


xgb_estimator.fit({
    "train": TrainingInput(train_input, content_type="csv"),
    "validation": TrainingInput(val_input, content_type="csv")
})
```

```
[04/02/25 04:27:26] INFO     Ignoring unnecessary Python version: py3.              image_uris.py:603
                    INFO     Ignoring unnecessary instance type: ml.m5.large.       image_uris.py:530
```

```
INFO     SageMaker Python SDK will collect telemetry to help us better   telemetry_logging.py:91
         understand our user's needs, diagnose issues, and deliver
         additional features.
         To opt out of telemetry, please disable via TelemetryOptOut
         parameter in SDK defaults config. For more information, refer
         to
         https://sagemaker.readthedocs.io/en/stable/overview.html#confi
         guring-and-using-defaults-with-the-sagemaker-python-sdk.
INFO     Creating training-job with name:                                        session.py:1042
         sagemaker-xgboost-2025-04-02-04-27-26-344
```

```
2025-04-02 04:27:28 Starting - Starting the training job...
..25-04-02 04:27:42 Starting - Preparing the instances for training.
..25-04-02 04:28:06 Downloading - Downloading input data.
..25-04-02 04:28:51 Downloading - Downloading the training image.
.[2025-04-02 04:29:31.953 ip-10-2-76-153.ec2.internal:7 INFO utils.py:28] RULE_JOB_STOP_SIGNAL_FILENAME: None
[2025-04-02 04:29:31.977 ip-10-2-76-153.ec2.internal:7 INFO profiler_config_parser.py:111] User has disabled profiler.
[2025-04-02:04:29:31:INFO] Imported framework sagemaker_xgboost_container.training
[2025-04-02:04:29:31:INFO] No GPUs detected (normal if no gpus installed)
[2025-04-02:04:29:31:INFO] Invoking user training script.
[2025-04-02:04:29:32:INFO] Module xgboost_train does not provide a setup.py.
Generating setup.py
[2025-04-02:04:29:32:INFO] Generating setup.cfg
[2025-04-02:04:29:32:INFO] Generating MANIFEST.in
[2025-04-02:04:29:32:INFO] Installing module with the following command:
/miniconda3/bin/python3 -m pip install .
Processing /opt/ml/code
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Building wheels for collected packages: xgboost-train
  Building wheel for xgboost-train (setup.py): started
  Building wheel for xgboost-train (setup.py): finished with status 'done'
  Created wheel for xgboost-train: filename=xgboost_train-1.0.0-py2.py3-none-any.whl size=3626 sha256=d6f76168516b2a6dee8ada06770
f07887413418ed6e0701b30202a18bc67c437
  Stored in directory: /home/model-server/tmp/pip-ephem-wheel-cache-juajazdb/wheels/3e/0f/51/2f1df833dd0412c1bc2f5ee56baac195b5be
563353d111dca6
Successfully built xgboost-train
Installing collected packages: xgboost-train
Successfully installed xgboost-train-1.0.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manage
r. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
[notice] A new release of pip is available: 23.0.1 -> 24.0
[notice] To update, run: pip install --upgrade pip
[2025-04-02:04:29:33:INFO] No GPUs detected (normal if no gpus installed)
[2025-04-02:04:29:34:INFO] Invoking user script
Training Env:
{
    "additional_framework_parameters": {},
    "channel_input_dirs": {
        "train": "/opt/ml/input/data/train",
        "validation": "/opt/ml/input/data/validation"
    },
    "current_host": "algo-1",
    "framework_module": "sagemaker_xgboost_container.training:main",
    "hosts": [
        "algo-1"
    ],
```

```
"hyperparameters": {},
"input_config_dir": "/opt/ml/input/config",
"input_data_config": {
    "train": {
        "ContentType": "csv",
        "TrainingInputMode": "File",
        "S3DistributionType": "FullyReplicated",
        "RecordWrapperType": "None"
    },
    "validation": {
        "ContentType": "csv",
        "TrainingInputMode": "File",
        "S3DistributionType": "FullyReplicated",
        "RecordWrapperType": "None"
    }
},
"input_dir": "/opt/ml/input",
"is_master": true,
"job_name": "sagemaker-xgboost-2025-04-02-04-27-26-344",
"log_level": 20,
"master_hostname": "algo-1",
"model_dir": "/opt/ml/model",
"module_dir": "s3://sagemaker-us-east-1-230393891640/sagemaker-xgboost-2025-04-02-04-27-26-344/source/sourcedir.tar.gz",
"module_name": "xgboost_train",
"network_interface_name": "eth0",
"num_cpus": 2,
"num_gpus": 0,
"output_data_dir": "/opt/ml/output/data",
"output_dir": "/opt/ml/output",
"output_intermediate_dir": "/opt/ml/output/intermediate",
"resource_config": {
    "current_host": "algo-1",
    "current_instance_type": "ml.m5.large",
    "current_group_name": "homogeneousCluster",
    "hosts": [
        "algo-1"
    ],
    "instance_groups": [
        {
            "instance_group_name": "homogeneousCluster",
            "instance_type": "ml.m5.large",
            "hosts": [
                "algo-1"
            ]
        }
    ],
```

```
        "network_interface_name": "eth0"
    },
    "user_entry_point": "xgboost_train.py"
}
Environment variables:
SM_HOSTS=["algo-1"]
SM_NETWORK_INTERFACE_NAME=eth0
SM_HPS={}
SM_USER_ENTRY_POINT=xgboost_train.py
SM_FRAMEWORK_PARAMS={}
SM_RESOURCE_CONFIG={"current_group_name":"homogeneousCluster","current_host":"algo-1","current_instance_type":"ml.m5.large","host
s":["algo-1"],"instance_groups":[{"hosts":["algo-1"],"instance_group_name":"homogeneousCluster","instance_type":"ml.m5.larg
e"}],"network_interface_name":"eth0"}
SM_INPUT_DATA_CONFIG={"train":{"ContentType":"csv","RecordWrapperType":"None","S3DistributionType":"FullyReplicated","TrainingInp
utMode":"File"},"validation":{"ContentType":"csv","RecordWrapperType":"None","S3DistributionType":"FullyReplicated","TrainingInpu
tMode":"File"}}
SM_OUTPUT_DATA_DIR=/opt/ml/output/data
SM_CHANNELS=["train","validation"]
SM_CURRENT_HOST=algo-1
SM_MODULE_NAME=xgboost_train
SM_LOG_LEVEL=20
SM_FRAMEWORK_MODULE=sagemaker_xgboost_container.training:main
SM_INPUT_DIR=/opt/ml/input
SM_INPUT_CONFIG_DIR=/opt/ml/input/config
SM_OUTPUT_DIR=/opt/ml/output
SM_NUM_CPUS=2
SM_NUM_GPUS=0
SM_MODEL_DIR=/opt/ml/model
SM_MODULE_DIR=s3://sagemaker-us-east-1-230393891640/sagemaker-xgboost-2025-04-02-04-27-26-344/source/sourcedir.tar.gz
SM_TRAINING_ENV={"additional_framework_parameters":{},"channel_input_dirs":{"train":"/opt/ml/input/data/train","validation":"/op
t/ml/input/data/validation"},"current_host":"algo-1","framework_module":"sagemaker_xgboost_container.training:main","hosts":["alg
o-1"],"hyperparameters":{},"input_config_dir":"/opt/ml/input/config","input_data_config":{"train":{"ContentType":"csv","RecordWra
pperType":"None","S3DistributionType":"FullyReplicated","TrainingInputMode":"File"},"validation":{"ContentType":"csv","RecordWrap
perType":"None","S3DistributionType":"FullyReplicated","TrainingInputMode":"File"}},"input_dir":"/opt/ml/input","is_master":tru
e,"job_name":"sagemaker-xgboost-2025-04-02-04-27-26-344","log_level":20,"master_hostname":"algo-1","model_dir":"/opt/ml/model","m
odule_dir":"s3://sagemaker-us-east-1-230393891640/sagemaker-xgboost-2025-04-02-04-27-26-344/source/sourcedir.tar.gz","module_nam
e":"xgboost_train","network_interface_name":"eth0","num_cpus":2,"num_gpus":0,"output_data_dir":"/opt/ml/output/data","output_di
r":"/opt/ml/output","output_intermediate_dir":"/opt/ml/output/intermediate","resource_config":{"current_group_name":"homogeneousC
luster","current_host":"algo-1","current_instance_type":"ml.m5.large","hosts":["algo-1"],"instance_groups":[{"hosts":["algo-
1"],"instance_group_name":"homogeneousCluster","instance_type":"ml.m5.large"}],"network_interface_name":"eth0"},"user_entry_poin
t":"xgboost_train.py"}
SM_USER_ARGS=[]
SM_OUTPUT_INTERMEDIATE_DIR=/opt/ml/output/intermediate
SM_CHANNEL_TRAIN=/opt/ml/input/data/train
SM_CHANNEL_VALIDATION=/opt/ml/input/data/validation
PYTHONPATH=/miniconda3/bin:/:/miniconda3/lib/python/site-packages/xgboost/dmlc-core/tracker:/miniconda3/lib/python37.zip:/minicon
```

```
da3/lib/python3.7:/miniconda3/lib/python3.7/lib-dynload:/miniconda3/lib/python3.7/site-packages
Invoking script with the following command:
/miniconda3/bin/python3 -m xgboost_train

2025-04-02 04:29:52 Uploading - Uploading generated training model
2025-04-02 04:30:11 Completed - Training job completed
Training seconds: 125
Billable seconds: 125
```

Shutdown Kernel

In [1]:

```
%%html
<p><b>Shutting down your kernel for this notebook to release resources.</b></p>
<button class="sm-command-button" data-commandlinker-command="kernelmenu:shutdown" style="display:none;">Shutdown Kernel</button

<script>
try {
    els = document.getElementsByClassName("sm-command-button");
    els[0].click();
}
catch(err) {
    // NoOp
}
</script>
```

Shutting down your kernel for this notebook to release resources.